Libra: Efficient Resource Management for Agentic RL Post-Training

Kaiwen Chen[1], Xin Tan[1], Jingzong Li[2], Zhi Zhou[3], Cen Li[3], Jiang Liu[3], Jie Meng[3], Jiazhi Jiang[1], Hong Xu[1]
[1]The Chinese University of Hong Kong
[2]The Hang Seng University of Hong Kong
[3]Independent Researcher

arXiv:2606.03077v3 [cs.LG] 16 Sep 2026

Abstract

Reinforcement learning (RL) has emerged as a standard post-training paradigm for shaping large language models (LLMs) into capable agents. In agentic RL, the rollout stage generates trajectories while invoking tools, producing long-tailed and non-stationary workloads that expose two fundamental challenges. First, due to the long-tailed response distribution, a small fraction of trajectories dominates rollout makespan. Second, rollout and training differ in their compute patterns, memory demands, and sensitivity to sequence length. As the policy evolves, shifts in the workload distribution further change their relative resource demands, making it difficult to maintain balanced execution across the two stages.

We present Libra, an adaptive runtime for agentic RL post-training with two complementary components: (1) intra-stage scheduling via a Causality-Guided Bucket Scheduler that routes requests across execution buckets with different parallelism configurations, reducing delays from rollout stragglers; and (2) cross-stage coordination that dynamically reallocates workers between rollout and training as the workload changes. It moves workers between the two stages through a non-blocking protocol without interrupting ongoing training. Evaluated on a 48× NVIDIA A800 GPU cluster and a 160× Ascend 910B3 NPU cluster across three agentic benchmarks, Libra achieves up to 4.2× higher throughput and up to 2.7× faster reward convergence.

CCS Concepts: • **Computer systems organization**;

Keywords: Agentic RL, RL post-training, Resource management

1 Introduction

Reinforcement learning (RL) has become an important post-training paradigm for large language models (LLMs), extending from alignment with human preferences [5, 8, 33] and complex reasoning [18, 26, 44] to the development of agentic capabilities [9, 10, 17, 23, 41, 54, 58]. A typical agentic RL iteration consists of three stages: trajectory generation (rollout), trajectory evaluation, and policy update (training). During rollout, the policy model generates trajectories through autoregressive decoding and interactions with external tools. During training, the collected trajectories and their rewards are used to compute advantages and update the policy. Trajectory evaluation assigns rewards to generated trajectories and is comparatively lightweight [18, 24, 37], leaving rollout and training as the dominant computational stages [13, 39, 63]. Overall pipeline throughput therefore depends primarily on how quickly rollout produces trajectories and training consumes them.

During agentic RL post-training, the model invokes external tools and incorporates their outputs into subsequent generation. The resulting workload has three important properties. First, trajectories are generated online so their lengths are determined at runtime. Second, tool invocations cause sequence lengths to fluctuate widely: sequence length varies substantially with tool execution outcomes, such as payload size and return status. Third, the sequence length distribution shifts continuously as the policy improves, making the workload non-stationary.

Efficiently executing this workload requires addressing two challenges. The first is mitigating the impact of long-tailed trajectories on rollout performance. Although most trajectories are short, a small fraction are substantially longer and can dominate the rollout makespan. Existing systems largely attempt to mitigate the long-tailed problem through upfront length prediction at $T = 0$ to classify requests and apply differentiated execution strategies. However, their prediction mechanisms become less reliable in agentic RL. Methods that exploit intra-group or cross-epoch response similarity [20, 35] assume that sequence length is largely prompt-determined, an assumption that breaks down in agentic RL where mid-trajectory tool execution outcomes heavily alter sequence length. Other approaches deploy auxiliary pre-trained language models or embedding-based predictors [59, 63]; however, as the actor policy continuously updates during RL post-training, these parametric predictors suffer severe representation drift and require frequent, compute-intensive retraining on GPUs.

The second is the cross-stage coordination challenge. Rollout and training act as networked peers that continuously exchange state: training propagates updated weights to rollout, while rollout streams generated trajectories back to training. However, they differ fundamentally in compute patterns, memory demands, and sensitivity to sequence length. Rollout

is memory- and bandwidth-bound and scales linearly with sequence length, whereas training is compute-bound and amortizes length variation through batching. Compounding the problem, the sequence length distribution drifts continuously as the policy evolves, rendering any static resource allocation progressively suboptimal. Existing RL frameworks either optimize rollout in isolation under static stage partitions [13, 39, 40] or merely reallocate GPU counts across rigid parallelism boundaries [49], failing to align resources with sequence-length demands. Meanwhile, distributed parallelization frameworks [32, 42, 61] target LLM pre-training, which are not designed to model the closed-loop state coupling, asymmetric scaling, and non-stationary drift inherent to RL post-training.

To address these challenges, we present Libra, an adaptive runtime designed for the disaggregated, asynchronous pipeline of agentic RL post-training. Libra treats rollout and training as co-designed, networked peers, establishing adaptive execution across two complementary levels: (1) intra-stage scheduling that performs causality-driven request routing across heterogeneous TP buckets, and (2) cross-stage coordination that coordinates resources and elastically rebalances workers across rollout and training over high-speed interconnects.

Intra-stage scheduling: To eliminate rollout stragglers, our first insight is that long-tailed problem can be effectively mitigated by matching requests to differentiated Tensor Parallelism (TP) configurations. A small TP degree minimizes cross-GPU communication overhead and maximizes token throughput for short sequences, whereas a large TP degree alleviates KV-cache memory pressure and sustains high decode efficiency for long sequences. Partitioning the rollout cluster into heterogeneous execution buckets with varying TP degrees (e.g., TP-1 to TP-8) allows requests of different sequence lengths to run under their optimal parallelism configurations, substantially reducing the straggler effect of long-tailed trajectories. However, dynamically steering requests to matching TP buckets requires timely length estimation. Here, Libra makes a second key observation: in agentic RL, tool execution outcomes serve as runtime signals that are informative of remaining sequence length. For example, a failed tool call triggers diagnostic retry chains that predictably inflate sequence length, while a massive tool return payload deterministically consumes substantial context. Building on these insights, Libra introduces the Causality-Guided Bucket Scheduler (§5). The scheduler tracks execution history using a lightweight causality-aware prefix tree (trie). Each node is keyed by the observed sequence of composite tool states (tool types, payload size categories, and success/failure statuses) and stores an empirical histogram of residual sequence lengths. Upon each tool return, the scheduler performs an $O(1)$ trie lookup to retrieve the residual length distribution and casts routing as an online cost-benefit decision.

Cross-stage coordination: At the cluster level, Libra dynamically maintains balanced execution across rollout and training under continuous workload drift (§4). Under a fixed cluster GPU budget, a global resource planner jointly optimizes both the GPU allocation across stages and the parallelism strategy within each stage. Guided by an analytical Cost Evaluator (CE), the planner explores the multi-dimensional training strategy space ((TP, EP, PP, DP)) via a topology-aware pruned decision tree, while determining optimal rollout heterogeneous bucket capacities through a dynamic programming formulation.

To enact these reallocations without interrupting ongoing execution, Libra organizes the cluster into three pools: a Core Training Pool, a Core Rollout Pool, and an Elastic Hybrid Pool. Libra recognizes that elasticity need not trigger global communicator rebuilds: by enforcing that elastic resources enter or exit strictly at the granularity of complete, self-contained Data-Parallel (DP) replicas, the core training topology remains immutable. Libra decouples communication into permanent intra-replica collective domains and a dynamic inter-replica RDMA control plane. Through a non-blocking joining protocol, hybrid DP replicas transition from rollout to training. They fetch model and optimizer snapshots asynchronously via RDMA during active training forward/backward steps, aligning optimizer moments at step boundaries without stalling ongoing training. Transient imbalances are absorbed immediately by hybrid workers toggling between rollout inference and training DP replicas, while persistent distribution shifts trigger planned repartitioning of core pools during inherent pipeline slack windows.

We evaluate Libra on both a 48-GPU cluster of NVIDIA A800s and a 160-NPU cluster of Ascend 910B3s across three representative agentic RL benchmarks: Search-R1 [25], R2E-Gym [22], and DAPO-Math-17K [57].

Libra consistently outperforms state-of-the-art baselines, achieving up to 4.2× higher pipeline throughput and 2.7× faster reward convergence. Libra has been open-sourced on https://github.com/NetX-lab/Libra.

In summary, this paper makes three main contributions:

- **Intra-stage Causality-Guided Scheduling**: We propose the Causality-Guided Bucket Scheduler, which leverages mid-trajectory tool returns as runtime signals within a prefix-tree structure to predict residual sequence lengths, routing requests across heterogeneous TP buckets.
- **Cross-stage Joint Planning and Elastic Coordination**: We design a global planner that jointly optimizes cross-stage resource splits and intra-stage parallelism strategies, coupled with an elastic execution mechanism that rebalances workers between rollout and training via decoupled communicators and a non-blocking joining protocol.
- **Implementation and Comprehensive Evaluation**: We implement Libra and evaluate it on 48× NVIDIA A800 GPUs and 160× Ascend 910B3 NPUs, demonstrating up

Case	Tool-call Count	Tool Tokens	Failure Num	Total Tokens	Label
django 14787	8	1,029	0	7,363	Small Payload
scikit-learn 11310	9	12,461	0	21,220	Large Payload
django 10999	46	3,962	3	40,960	Failure Cascade
django 16560	50	7,809	8	40,960	Failure Cascade

Table 1. Trajectory divergence across tasks in R2E-Gym [22], generated by Qwen3-14B.

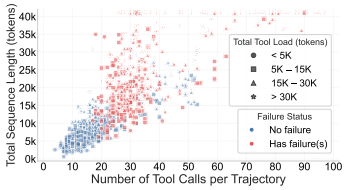


Figure 1. Tool-call count versus sequence length in R2E-Gym [22], generated by Qwen3-14B.

to 4.2× throughput speedup and 2.7× faster reward convergence across diverse agentic workloads.

2 Background and Motivation

We start by presenting the background and motivation.

2.1 Disaggregated and Asynchronous RL

Recent RL frameworks commonly execute rollout and training in a disaggregated and asynchronous manner [13, 46, 47, 50] to better accommodate the distinct execution characteristics of each stage. The memory- and bandwidth-bound rollout runs on a separate cluster of GPU workers, which continuously generate trajectories with slightly stale weights, while the compute-intensive training cluster periodically pushes updated parameters back to the rollout side [13, 15, 16, 19, 39, 46, 63].

2.2 Characteristics of Agentic RL Post-Training

Agentic RL post-training workloads exhibit four properties:

Long-tailed Rollout. A large body of work [14, 35, 39, 59, 63, 64] has identified the long-tailed output sequence as a major source of rollout inefficiency. Although most trajectories conclude within a short reasoning process, a small fraction is substantially longer. These long-tailed sequences dominate rollout makespan, forcing faster-finishing workers to idle while waiting for the slowest trajectories to complete. According to our measurements on R2E-Gym [22] (Qwen3-14B), the longest 10% trajectories account for over 50% of total rollout time.

Causal Drivers of Sequence Length. To our knowledge, we are the first to identify and systematically characterize this phenomenon: In agentic RL, sequence length is causally driven by external tool invocations at runtime. Tool-return payload size and success/failure status are the principal causal drivers of trajectory expansion, introducing substantial runtime variability beyond the initial prompt. Table 1 demonstrates this causal relationship across representative tasks in R2E-Gym [22]. First, tool-return payload size directly dictates context expansion: a lightweight payload (django-14787, 1K tool tokens) yields a concise trajectory of < 8K tokens, whereas an extensive tool return (`scikit-learn-11310`, 12.5K tool tokens) abruptly inflates the sequence to over 21K tokens. Second, tool execution status acts as an even more decisive causal trigger: when tool calls fail, the model enters diagnostic retry loops and error-correction steps. As shown in `django-10999` and `django-16560`, such failure cascades multiply tool invocations by over 5×, driving total length to the 41K context ceiling. Figure 1 corroborates this causal link across the full benchmark: failed trajectories systematically concentrate in the high-call, extreme-length regime, while successful trajectories remain in lower-length tiers.

Non-stationary Nature of the Workload. As RL post-training proceeds, the actor model's reasoning capability is continuously updated, which alters its response patterns gradually. Figure 2(b) illustrates one manifestation of this drift: on our agentic RL benchmark, the average sequence length grows from ∼2,500 to ∼11,500 tokens, causing rollout time to gradually overtake training time. We note that the direction and magnitude of length drift are not fixed—they depend on the model architecture, the RL algorithm, and the task domain (e.g., some workloads may exhibit length contraction rather than expansion [44, 45, 52]). What is guaranteed, however, is that the sequence length distribution shifts as the policy evolves, so any statically chosen resource split progressively diverges from the optimal allocation.

Strong Asymmetry Between Rollout and Training. Figure 2(a) shows that when sequence length grows from 1K to 32K tokens, rollout latency increases by 95× (from 30 s to 2,850 s), whereas training time increases by only 3.9× (from 135 s to 527 s). Rollout latency grows tremendously with sequence length due to autoregressive decoding, while training amortizes length variation through batching.

2.3 Opportunities

These characteristics motivate us to optimize agentic RL post-training along two complementary axes.

Rollout Optimizations. Rollout efficiency in agentic RL can be improved by two ideas. First, the sharp divergence in sequence length motivates us to consider heterogeneous parallelism configurations, especially Tensor Parallelism (TP), tailored to different sequence lengths. Figure 3 shows that TP-1 achieves 1,852 token/s for short sequences (0k–2k) but collapses to 430 token/s at 16k–32k due to KV-cache pressure; conversely, TP-8 starts at only 591 token/s for short sequences but sustains 1,220 token/s at 16k–32k, outperforming TP-1 by 2.8× in our experiment. Large TP partitions the KV cache and model weights across more GPUs, mitigates memory bottlenecks, and amortizes the all-reduce overhead over heavier computation for long sequence; small TP reduces communication overheads, matching the low memory needs profile of short requests. Thus, rather than adopting a uniform TP configuration across the rollout cluster, we concurrently deploy multiple TP configurations within the cluster.

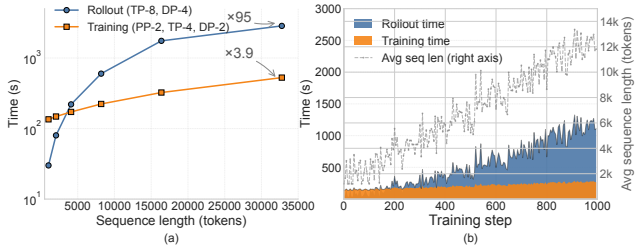


Figure 2. (a) Comparison of average latency between rollout and training as output sequence length increases. (b) Workload drift over the course of training. We use Qwen3-32B-Base on A800 80GB GPUs; the rollout stage runs on 32 GPUs with TP-8 and DP-4 (batch size 512), and the training stage runs on 16 GPUs with PP-2, TP-4, and DP-2 (global batch size 4096, mini-batch size 16) on AIME [30].

The cluster is partitioned into buckets, where each bucket hosts inference instances with a distinct TP degree optimized for requests within certain ranges of sequence lengths. We focus on TP because Pipeline Parallelism (PP) can exacerbate straggler effects through pipeline bubbles under variable-length trajectories, and Expert Parallelism (EP) addresses expert load balancing rather than sequence-length-induced stragglers. Further details are discussed in Appendix A.

The next natural idea is request-bucket assignment based on runtime causal signals. A straightforward approach is to predict each trajectory's final length and dispatch it to the matching bucket using a dedicated model [38, 59]. However, as the policy drifts during the course of training, the prediction accuracy will be degraded without periodical retraining. An alternative is prediction-free heuristics like the classical MLFQ, which promotes requests to larger-TP buckets only after their lengths cross fixed thresholds. It is both delayed and incremental: trajectories suffer latency at ill-matched configurations before any threshold is crossed, and long trajectories endure multiple migrations with each incurring state-transfer overhead before reaching their final bucket.

The Causality-Guided Bucket Scheduler exploits the causal signals from the tool-calling results to make routing decisions. A trajectory encountering a heavy payload or a tool-execution failure can be promoted to a larger TP bucket because its sequence length is very likely to expand, while one with lightweight tool returns may remain on its current setup. This assignment is dynamic during the request's lifetime and resolves the limitations of both baselines. First, Libra naturally adapts to policy drift: when RL optimization shifts the trajectory distribution, Libra only needs to update the decision tree offline instead of retraining a prediction model. Second, Libra enables earlier and decisive migration: whereas MLFQ upgrades a request only after its length crosses a threshold, our bucket scheduler makes a routing decision immediately upon each tool return, often migrating a request to its final bucket in a single step rather than through gradual promotion.

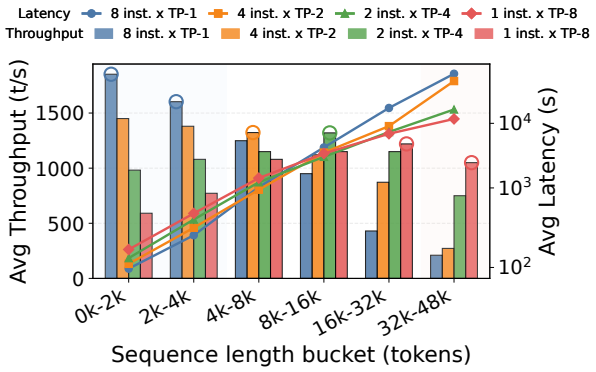


Figure 3. Average throughput vs. sequence length across TP configurations on 8xA800 GPUs (Qwen3-14B, batch size 512, R2E-Gym). "2 inst. x TP-4" denotes two instances each using 4-GPU Tensor Parallelism.

Cross-Stage Optimizations. Since rollout and training execute concurrently, the end-to-end iteration time is dictated by whichever stage runs slower under the current GPU allocation. The bottleneck is a dynamic consequence, not a fixed property of rollout. This exposes a clear opportunity: by dynamically reallocating GPUs from the faster stage to the slower one, the system can mitigate the bottleneck and reduce the overall iteration time. However, the optimal allocation is not static, as the workload is non-stationary and the balance point drifts over time. A system that continuously tracks this drift and uses lightweight elasticity to reassign GPUs without disrupting execution can sustain higher throughput than any static allocation.

3 Libra Overview

Libra is an adaptive runtime for the disaggregated, asynchronous pipeline of agentic RL post-training. To address the long-tailed and cross-stage coordination challenges identified in previous sections, Libra organizes its execution around a two-level adaptive architecture (Figure 4).

Intra-Stage Scheduling (§5). At the request level, Libra partitions the rollout cluster into heterogeneous TP buckets and routes each request to the bucket matching its predicted residual length. Routing is driven by the Causality-Guided Bucket Scheduler, which treats tool-execution outcomes as runtime signals for residual-length prediction, eliminating rollout stragglers without auxiliary predictors.

Cross-Stage Coordination (§4). Operating at the cluster level, cross-stage coordination maintains balance across the disaggregated pipeline. A global planner periodically resolves the cross-stage worker allocation and per-stage parallelism strategy under the current workload, guided by an analytical Cost Evaluator. Reallocations are enacted through two tiers: (i) a lightweight tier where hybrid workers dynamically transition between rollout inference and training

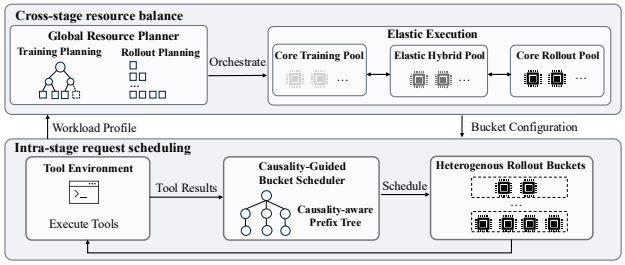


Figure 4. Libra overview

data-parallel replicas via decoupled communication domains and a non-blocking RDMA joining protocol, resolving transient bottlenecks without stalling ongoing training; and (ii) a heavier tier that rebuilds the core pools' parallelism strategy during pipeline slack periods when the workload shift proves persistent.

4 Global Resource Planner with Elastic Execution

In an asynchronous RL pipeline, the iteration time is bounded by the slower stage: $T_{\text{iter}} = \max(T_{\text{rollout}}, T_{\text{train}})$. The resource configurations of the training and rollout clusters are inherently coupled. Allocating more GPUs to training reduces T_{train} but leaves fewer GPUs for rollout, increasing T_{rollout} and restricting the range of viable heterogeneous TP configurations; symmetrically, over-provisioning the rollout cluster starves training of resources, raising T_{train} and shifting the bottleneck to training. Consequently, local optimization leads to stage starvation. To maximize end-to-end throughput under a fixed GPU budget (N_{GPU}), the system must jointly optimize both clusters from a global perspective.

Libra addresses this coupled allocation problem with a hierarchical planner that decomposes the joint search space into training strategy selection and rollout partitioning to minimize the iteration makespan. The training side employs a topology-aware decision tree with pruning to enumerate feasible parallel strategies, reducing the candidate set from exponential size to dozens. The rollout side replaces exhaustive integer partitioning with a dynamic programming formulation. This section details these two components in turn, and further describes how Libra realizes the planner's reallocations without global reconfiguration through the Elastic Hybrid Pool.

4.1 Training Side: Decision Tree-Based Parallel Strategy Enumeration

The training strategy search space $\mathcal{S}$ comprises all valid (TP, PP, DP) factorizations of n_{train} for dense models. For MoE models, the space expands to (TP, EP, PP, DP), where EP denotes Expert Parallelism. A naive enumeration of this space scales exponentially in the number of divisors of n_{train}. Libra instead constructs a topology-aware decision tree that

Algorithm 1 Global Resource Planner

def resourcePlanner(N_{GPU}, L, M_{limit}, $\mathcal{T}$, CE):
  $T^*_{\text{iter}} = \infty$
  MemoRollout = $\emptyset$
  for $n_{\text{train}} = 1$ **to** N_{GPU} **do**
    $\mathcal{S}_{\text{cand}} = \text{DecisionTreeSearch}(n_{\text{train}}, M_{limit}, \text{CE})$
    if $\mathcal{S}_{\text{cand}} = \emptyset$ **then continue**
    $s_{\text{best}} = \arg\min_{s \in \mathcal{S}_{\text{cand}}} \text{CE.TrainTime}(s)$
    $T_{\text{train}} = \text{CE.TrainTime}(s_{\text{best}})$
    $n_{\text{rollout}} = N_{GPU} - n_{\text{train}}$
    if $n_{\text{rollout}} \notin$ MemoRollout **then**
      MemoRollout$[n_{\text{rollout}}] = \text{RolloutDP}(n_{\text{rollout}}, L, \mathcal{T}, \text{CE})$
    $T_{\text{rollout}} = \text{MemoRollout}[n_{\text{rollout}}]$
    $T_{\text{max}} = \max(T_{\text{train}}, T_{\text{rollout}})$
    if $T_{\text{max}} < T^*_{\text{iter}}$ **then**
      $T^*_{\text{iter}} = T_{\text{max}}$, record $(n_{\text{train}}, s_{\text{best}})$
  return optimal config

def RolloutDP(n_{rollout}, L, $\mathcal{T}$, CE):
  Sort requests by length; let indices be $1 \ldots L$
  Init $dp[g][i] = +\infty$ for $0 \le g \le n_{\text{rollout}}, 0 \le i \le L$
  $dp[0][0] = 0$
  for $g = 1$ **to** n_{rollout} **do**
    for $i = 0$ **to** L **do**
      for $tp \in \mathcal{T}$ **with** $tp \le g$ **do**
        for $x = 0$ **to** i **do**
          $t_{\text{inst}} = \text{CE.RolloutTime}(tp, i - x + 1, i)$
          $t_{\text{cand}} = \max(dp[g - tp][i - x], t_{\text{inst}})$
          $dp[g][i] = \min(dp[g][i], t_{\text{cand}})$
  return $dp[n_{\text{rollout}}][L]$

enumerates candidates level by level, pruning infeasible and inefficient branches at each depth. The tree is extensible, using three levels for dense models and four levels for MoE models, with an additional EP level inserted after TP.

Decision tree structure. The tree is rooted at n_{train} and grows through parallel-strategy levels. At the first level, TP is restricted to $\{1, 2, 4, 8\}$, reflecting the single-node GPU count and the NVLink connectivity domain. Branches are pruned if the per-GPU memory footprint exceeds M_{limit} or if the projected communication-to-computation ratio exceeds a configurable threshold α. For MoE models, the EP level enumerates expert-parallel degrees that divide the total expert count and satisfy $TP \times EP \le n_{\text{train}}$. Branches are pruned if their all-to-all communication volume exceeds a configurable threshold β, measured relative to the compute time of the corresponding MoE layers. Placing EP before PP maximizes the likelihood of keeping all-to-all communication within a single node, which is critical for MoE performance. At the next level, PP candidates are integers satisfying $TP \times PP \le n_{\text{train}}$ for dense models, or $TP \times EP \times PP \le n_{\text{train}}$ for MoE models. Branches are pruned if their pipeline bubble ratio $\frac{PP-1}{PP+m-1}$, where m denotes the number of micro-batches, exceeds a configurable threshold (e.g., 30%). At the final level, $DP = n_{\text{train}}/(TP \times PP)$ for dense models, or $DP = n_{\text{train}}/(TP \times EP \times PP)$ for MoE models. A path is valid only when this division yields an integer. Each root-to-leaf path defines a complete training

strategy s_{train}. Libra evaluates $T_{\text{train}}(s_{\text{train}})$ for every surviving strategy.

4.2 Inference Side: Dynamic Programming for Optimal Heterogeneous Partitioning

Given n_{rollout} GPUs and a total workload of L samples, the rollout subproblem partitions the GPUs into heterogeneous inference instances, each with a tensor-parallel degree $tp \in \mathcal{T}$ (where $\mathcal{T} = \{1, 2, 4, 8\}$), and assigns a subset of the workload to each instance. The objective is to minimize the makespan, i.e., the maximum completion time across all instances. This problem exhibits optimal substructure and admits an efficient dynamic-programming solution, eliminating the need for brute-force enumeration over integer partitions.

Cost interface. The underlying cost evaluator provides a function $\text{Cost}(tp, a, b)$ that returns the minimum time for a single inference instance with tensor parallelism tp to process the contiguous segment of requests indexed from a to b in the sorted list. This function internally accounts for maximum batch size limits, dynamic pipeline bubbles, and variable-length sequences, encapsulating all low-level execution complexity.

DP formulation and solving procedure. Prior to the DP, all L requests are sorted by their sequence length in non-decreasing order and indexed $1, 2, \ldots, L$. The DP constructs a heterogeneous partition by incrementally adding inference instances. Each instance is characterized by a configuration (tp, x), where $tp \in \mathcal{T}$ is its tensor-parallel degree and x is the number of samples it serves. Because different instances may take different tp values, the resulting GPU partition forms a *heterogeneous TP group*. The DP enumerates all possible ways to append one such instance to an already-optimal sub-partition for the remaining GPUs and the remaining prefix of sorted samples, exploiting optimal substructure to solve the global makespan-minimization problem.

Concretely, let $dp[g][i]$ denote the minimum achievable makespan when using exactly g GPUs to serve the first i requests in the sorted list. We initialize $dp[0][0] = 0$ and set $dp[g][i] = +\infty$ for all other states. The state transition is:

$$dp[g][i] = \min_{\substack{tp \in \mathcal{T},\ tp \le g \\ 0 \le x \le i}} \max\left(dp[g - tp][i - x],\ \text{Cost}(tp, i - x + 1, i)\right)$$

The transition considers the last added instance with configuration (tp, x): it consumes tp GPUs, processes the contiguous segment of requests indexed $i - x + 1$ through i in the sorted list, and contributes $\text{Cost}(tp, i - x + 1, i)$ to the overall makespan. The inner max captures the bottleneck instance time, while the outer min selects the best partition. The optimal rollout time is $T^*_{\text{rollout}} = dp[n_{\text{rollout}}][L]$, and the optimal partition P^*_{rollout} is recovered by backtracking from this state.

Sample ordering by sequence length. Sequence length affects the optimal partition because large-tp instances better accommodate long sequences, whereas small-tp instances are more efficient for short sequences. Sorting the requests by sequence length *before* the DP traversal is essential for correctness: the DP state tracks how many requests from the sorted prefix have already been assigned, so every instance must receive a contiguous segment of the sorted list. Consequently, the DP naturally dispatches shorter requests to smaller-tp instances and longer requests to larger-tp instances, making the length-to-TP matching both correct and effective.

Length profiling. The sequence length of each prompt is drawn from historical rollout data. Prior to training, an initial profiling run establishes a baseline length for every prompt. Whenever a new rollout completes for a prompt, its stored length is updated with the latest observation, allowing the planner to adapt to distribution shifts over time.

Complexity reduction. The state space has size $O(n_{\text{rollout}} \cdot L)$. A naive transition enumeration over all tp and x would yield $O(n_{\text{rollout}} \cdot L^2 \cdot |\mathcal{T}|)$ complexity. This pseudo-polynomial complexity is still substantially smaller than exhaustive enumeration over all integer partitions of n_{rollout}, the partition function $p(n_{\text{rollout}})$ grows rapidly (e.g., $p(64) \approx 1.7 \times 10^6$). In practice, the rollout subproblem remains tractable because $|\mathcal{T}|$ is small and each rollout DP result is memoized for reuse by outer-loop states with the same rollout budget.

4.3 Hierarchical Search with Rollout Memoization

Algorithm 1 integrates the two subproblem solvers into a global nested search. The outer loop iterates over feasible values of n_{train} (e.g., 1 to N_{GPU}) and invokes the decision tree to obtain the set of valid training strategies $\mathcal{S}_{\text{cand}}$ for each budget, yielding the optimal training time $\min_{s \in \mathcal{S}_{\text{cand}}} T_{\text{train}}(s)$. The rollout DP is invoked once per distinct $n_{\text{rollout}} = N_{GPU} - n_{\text{train}}$ and its result is memoized in MemoRollout. Because different outer-loop states that share the same rollout budget reuse the memoized result, the planner avoids redundant DP executions and remains efficient even at large cluster scales. In Algorithm 1, CE denotes the cost evaluator, which provides the estimated training and rollout times used by the planner; its design is detailed in §4.5.

The planner is invoked periodically (e.g., every K training iterations) rather than once at initialization. The workload L is continuously updated as prompts are revisited during rollout; at each invocation, the planner reads the latest statistics and computes the new optimal configuration. If the projected throughput gain exceeds the reconfiguration cost, Libra triggers a resource reallocation.

Realizing these reallocations, however, requires bridging a gap between planning and execution: in conventional frameworks [1, 34, 60], adding or removing training workers forces a global communicator rebuild and state redistribution, making frequent reallocation impractical. Figure 5 illustrates an example of how Libra expands the training cluster by adding a hybrid worker without disrupting the core training pool.

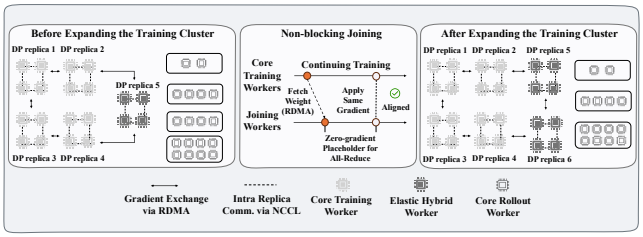


Figure 5. Elastic execution example.

4.4 Elastic Hybrid Pool

Libra's insight is that resource elasticity need not be a global reconfiguration event. If the core training topology is kept strictly immutable and workers enter or exit only as complete, unsharded data-parallel replicas, they can participate as *external* members rather than *internal* collective ranks.

Three-pool organization. To materialize these principles, Libra partitions the cluster into a Core Training Pool, a Core Rollout Pool, and an Elastic Hybrid Pool. The two core pools provide stable capacity for the dominant workload modes. The Core Training Pool performs synchronized weight updates and maintains static data-, tensor-, and pipeline-parallel groups. The Core Rollout Pool handles trajectory generation with the heterogeneous TP configurations determined by the planner. The Elastic Hybrid Pool provides fast-response capacity for resource reallocation. When rollout becomes the bottleneck, Hybrid workers switch to rollout mode to expand generation throughput. Conversely, when training becomes the bottleneck, they rejoin training as additional DP replicas. Because each Hybrid worker holds a complete, unsharded copy of model weights and optimizer states, its addition or removal does not reshape the tensor- or pipeline-parallel topology of the core training pool. Importantly, the Elastic Hybrid Pool need not be fully reserved in advance: because rollout instances are stateless, Libra can temporarily borrow workers from the Core Rollout Pool, convert them into Hybrid workers, and later return them with low transition cost. This mechanism covers transient training-side shortages without perturbing the core training topology. Rollout-side shortages are handled even more cheaply by switching existing Hybrid workers into rollout mode. If RL optimization induces a persistent demand shift, Libra can rebuild the core partition itself; such rebuilds are infrequent, because short-term imbalances are absorbed by the Elastic Hybrid Pool and large repartitioning can be deferred to slack periods—intervals in which training is already stalled waiting for fresh rollout data, which naturally serve as reconfiguration windows.

Decoupled communication domains. Building on non-blocking fault-tolerant training [36], Libra decouples intra-replica and inter-replica communication into independent planes. We define the intra-replica domain as the NCCL collective within a single data-parallel replica; this domain remains strictly stable and is never rebuilt. The inter-replica domain, conversely, handles traffic across data-parallel replica (e.g., gradient exchange between the core training pool and elastic hybrid workers). All elasticity-related membership changes are confined to the inter-replica domain, where an independent control plane manages topology rewiring and RDMA connection establishment without disturbing the data plane. Implementation details are provided in Appendix C.

Non-blocking joining. Transitioning a Hybrid worker from rollout back to training requires reloading model and optimizer states. If the active training cluster were to pause and wait for this restoration, it would suffer from severe synchronization overhead. Libra avoids this pause through a non-blocking joining protocol inspired by non-blocking fault-tolerant training [36]. At the end of each training step, active training workers asynchronously capture a snapshot of their current weights and optimizer states. When a Hybrid worker transitions back to training, it fetches this snapshot and joins as a DP replica. During the restoration window, the active training cluster advances to the next step without waiting.

Crucially, the joining worker does not directly join the core training pool's All-Reduce collective. Instead, it routes its gradient to the corresponding core training rank through the asynchronous side channel via RDMA. During recovery, this gradient is a zero placeholder. Because the core rank accumulates the external gradient into its local backward pass, a zero placeholder leaves the core rank's local gradient unchanged. Consequently, the intra-core All-Reduce produces exactly the same averaged gradient as training without the joining worker, preserving mathematical equivalence (Appendix D). Once the snapshot is loaded, the joining worker's parameters and optimizer state are overwritten by the snapshot, attaining identical state to the rest of the cluster without requiring iteration replay or global synchronization.

4.5 Underlying Cost Evaluator

The planner relies on an accurate cost evaluator (CE) to provide CE.TrainTime(s) and CE.RolloutTime(tp, a, b). However, existing simulators need to be adapted for RL post-training because the highly variable length of generated sequences causes severe workload imbalance. Inference simulators [2] emphasize general request serving and model operator execution via black-box estimation, which lacks the generalization capability needed for the extreme length variations in RL generation. Training simulators [32, 42] rely on a steady-state assumption, calculating a static pipeline beat based on uniform micro-batch lengths. This simplification misses the dynamic pipeline bubbles where downstream stages are forced to idle while waiting for the completion of a micro-batch containing long-tail sequences. To accurately reflect the physical execution complexity in RL post-training

scenarios, Libra's cost evaluator inherits mechanisms from existing simulators and modifies them to support dynamic length distributions. Please note that the CE is a modular design; other simulators [11, 12, 48] can also be adapted with modifications.

For the rollout phase, Libra inherits profiling-guided, operator-triaged runtime prediction from Vidur [2]. However, it replaces Vidur's random forest estimators with polynomial fitting—$O(L)$ for Linear operators and $O(L^2)$ for Attention operators. This replacement is necessary because polynomial functions provide stronger physical interpretability and robust generalization for the extreme sequence length variations in RL generation. Specifically, it formulates the execution time of each layer and stage as a direct function of the sequence length L, such as $T_{\text{Attn}}(L) = \gamma \cdot L^2 + \eta \cdot L + \theta$, rather than relying on black-box predictions like random forests. The CE.RolloutTime(tp, a, b) interface used by the DP internally invokes these per-operator models, aggregates them across layers and pipeline stages, and accounts for batching effects and dynamic load imbalance across the assigned segment of samples.

For the training phase, Libra inherits the execution graph simulation (e.g., 1F1B Pipeline Parallelism) and iteration time prediction framework from Sailor [42]. The primary modification is abandoning Sailor's steady-state assumption in favor of micro-bubble capture. This shift is required because the heterogeneous length distribution creates misaligned computational loads across micro-batches, meaning a uniform steady-state beat simply does not exist. Instead of multiplying a bottleneck stage duration by the number of micro-batches, Libra explicitly calculates the start and finish times for each micro-batch at each pipeline stage, taking into account data arrival and hardware availability. This non-uniform simulation accurately captures the bottleneck drift and accumulated micro-bubbles caused by length mismatch.

5 Causality-Guided Bucket Scheduler for Heterogeneous Rollout Clusters

We first present Libra's intra-stage scheduling, which addresses length heterogeneity and eliminates stragglers through Causality-Guided Bucket Scheduler across heterogeneous execution buckets. As motivated in Section 2.3, Libra instantiates the rollout cluster as a set of heterogeneous buckets $\mathcal{B} = \{B_1, B_2, \ldots, B_m\}$ ordered by increasing compute capacity, where each bucket contains inference instances with a distinct TP configuration. The global planner described in §4 provisions these buckets, and the bucket scheduler serves as the runtime scheduling policy that maps individual requests to them conditioned on the causal signals revealed by tool execution. Figure 6 illustrates the end-to-end Causality-Guided Bucket Scheduler workflow on a heterogeneous rollout cluster.

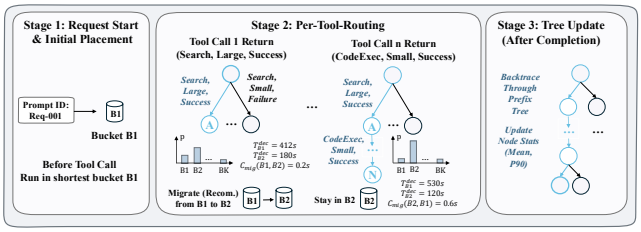


Figure 6. A scheduling example of Causality-Guided Bucket Scheduler.

Causality-aware prefix tree. To turn this causal intuition into a routing decision, Libra builds a lightweight prefix-tree (trie) structure from historical trajectories. Each node in the tree represents a unique prefix of the tool-call history, keyed by the ordered sequence of return states from all prior tool interactions. An edge corresponds to invoking a specific tool type and receiving its abstracted return state, which encodes the payload characteristics (e.g., size class) and the execution outcome (e.g., success or failure). Formally, a node at depth k is uniquely identified by the prompt identifier and the sequence of k observed return states, denoted $\kappa = (\texttt{prompt_id}, \langle s_1, s_2, \ldots, s_k \rangle)$, where each s_i denotes the composite state of the i-th tool call, comprising the tool type invoked as well as its return attributes such as payload size (SizeSmall, SizeLarge) and execution status (ToolSuccess, ToolFailure), derived from the environment response. Every node stores a histogram of the remaining sequence length distribution from that point onward, with bins aligned to the length thresholds of the buckets, for runtime routing. Libra builds the tree offline from trajectory logs (Appendix B): for each trace, it walks down the tree and records the remaining length at every visited node, enabling $O(1)$ runtime lookups of conditional residual length distributions.

Three-stage scheduling pipeline. Causality-Guided Bucket Scheduler uses the prefix tree to route requests through three phases.

Phase 1: Initial placement. Before the first tool call, all trajectories are placed in the shortest bucket. Because the initial prompt and early reasoning contents typically occupy limited context, assigning every request to a high-TP instance would reduce cluster-wide utilization.

Phase 2: Per-tool-return routing. When the model emits a tool-call token, Libra pauses decoding, offloads the request from the GPU, and waits for the external environment to execute the tool asynchronously. After the tool returns, the scheduler extracts the return-state label and walks down the prefix tree from the current node. At the reached node, it retrieves the empirical histogram $F = \{p_1, p_2, \ldots, p_K\}$ of the remaining trajectory length, where p_k is the probability mass in bin k. The bins are aligned with the buckets's thresholds; Causality-Guided Bucket Scheduler then casts routing as a cost-benefit decision: for each candidate bucket $B \in \mathcal{B}$,

it estimates the expected remaining decode time as $T_B^{\text{dec}} = \sum_{k=1}^{K} p_k \cdot \bar{T}_{\text{dec}}(B,k)$, where $\bar{T}_{\text{dec}}(B,k)$ is the decode cost for bucket B on bin k (using the planner's cost model, described in §4.5), and selects

$$B^* = \arg\min_{B \in \mathcal{B}} \Big(\sum_{k=1}^{K} p_k \cdot \bar{T}_{\text{dec}}(B,k) + C_{\text{mig}}(B_{\text{cur}}, B) \Big),$$

where $C_{\text{mig}}(B_{\text{cur}}, B)$ is the cost of moving the request's KV state to B, and $C_{\text{mig}}(B_{\text{cur}}, B_{\text{cur}}) = 0$. It is profiled per network path as the cheaper of transfer and recomputation (detailed below). If the walk reaches an unseen node, the scheduler backs off to its parent's distribution.

Phase 3: Tree update. Offline, Libra incrementally updates the tree by inserting newly collected traces along their return-state sequences (Appendix B.2), recording residual lengths at visited nodes to continuously align empirical distributions with policy drift.

This design yields three advantages: routing decisions require only cheap $O(1)$ lookups without online model inference, policy drift is tracked via lightweight offline tree updates without parametric model retraining, and requests migrate directly to target buckets in a single step upon tool returns rather than through incremental promotion.

Migration cost and system behavior. Libra migrates a request across buckets in three steps, each of which incurs at most negligible overhead.

Step 1: KV-cache offload during tool execution. When the model emits a tool-call token, the source GPUs offload the request's KV cache to CPU pinned memory through PCIe. This offload is not migration-specific: it is standard practice in production inference systems to offload a paused request's KV cache to CPU memory so that concurrent decoding requests can proceed efficiently [6, 28, 51]. Libra performs the same offload whether the request will later migrate or resume in place, so the PCIe transfer time can be masked by the asynchronous tool I/O and never appears on the scheduling critical path.

Step 2: CPU-side resharding. While the KV cache resides in CPU memory, Libra reshards it along the attention-head dimension to match the target bucket's TP configuration. If the target TP is larger ($\text{TP}_{\text{target}} > \text{TP}_{\text{source}}$), each source partition is split into $\frac{\text{TP}_{\text{target}}}{\text{TP}_{\text{source}}}$ shards; if the target TP is smaller, every $\frac{\text{TP}_{\text{source}}}{\text{TP}_{\text{target}}}$ contiguous partitions are concatenated into one. Both operations are pure CPU memory rearrangements with no data movement across buses, so their latency is negligible.

Step 3: GPU reload at tool return. When the tool result returns and the target bucket is determined, each target GPU loads its own shard from CPU memory and resumes decoding. If the source and target buckets reside on the same node, the reload latency is essentially identical whether the request moves to another GPU or returns to the same one.

For cross-node migration, the KV cache must traverse the inter-node network before decoding can resume. Libra therefore compares the end-to-end cost of transferring the KV cache against the cost of recomputing it from scratch on the target bucket. This comparison accounts for the full migration pipeline: PCIe staging, CPU-side resharding, network transfer, and target-GPU reload. Libra builds the decision on empirically measured latency profiles. Offline micro-benchmarks characterize the migration latency (including all staging overheads) and the recomputation prefill latency for each target TP configuration across the relevant range of sequence lengths. At runtime, Libra queries this profile to select the cheaper path for each request. As shown in Section 7.4, the measured end-to-end migration latency stays below 733 ms even at 40K-token prefixes, and the crossover point where recomputation becomes cheaper falls around 4K tokens for typical configurations.

6 System Implementation

Libra is implemented in approximately 13,000 lines of Python and C++/CUDA code. The core asynchronous RL training pipeline is built on top of verl [40] (v0.8.0), using vLLM [27] (v0.9.2 on CUDA; v0.22.1 on Ascend NPU) for generation and Megatron-Core [1] (v0.14.0, with `megatron-bridge` v0.2.0rc6) for policy training under PyTorch 2.7.0 (torch-npu 2.7.1). We further describe the implementation of some of the unique aspects of Libra.

Decoupled communication domains. To enable elastic execution, Libra separates control plane and data plane in cross-replica gradient exchange. The CPU manages all complex control logic: adding/removing replicas, RDMA connection re-establishment, timeout handling, and congestion control via fixed-size chunk pipelining (16 MB chunks). The GPU only executes data copy and reduction kernels using a ring algorithm (ReduceScatter + AllGather). This hybrid design allows us to reconfigure communication groups without restarting core training pool. The CPU thread coordinates with a consensus service to detect failures and determine the active replica set; then it updates RDMA send/receive buffers accordingly. Meanwhile, the GPU stream busy-polls a host-pinned flag—when data arrives, it performs in-GPU reduction. The kernel is optimized to use only 2 SMs (vs. NCCL's 4) by exploiting instruction-level parallelism and register reduction, achieving comparable bandwidth while avoiding contention with computation (Appendix C).

Non-blocking joining. Elastic workers fetch the latest checkpoint directly from the core training pool via a load-balanced P2P RDMA protocol over CPU host memory, which runs over host memory and does not interfere with GPU high-speed network. To join without stalling, Libra leverages a zero-gradient trick: the recovering replica sends a zero gradient through its side channel to the corresponding core rank instead of its computed one. Because the core rank's local gradient is unchanged by this zero placeholder, the subsequent intra-core All-Reduce remains mathematically

equivalent to training without the recovering replica. Once the snapshot is loaded, all replicas resume from identical parameter and optimizer states (Appendix D).

7 Evaluation

Setup. We deploy Libra on two clusters. The first is a GPU cluster of 48 NVIDIA A800-SXM4-80GB GPUs (6 nodes, 8 GPUs per node), connected via NVLink (NVSwitch, 600 GB/s bidirectional) and a 200 Gb/s RoCE (RDMA over Converged Ethernet) fabric with GPUDirect RDMA. The second is an NPU cluster of 160 Ascend 910B3 NPUs (20 nodes, 8 NPUs per node, 64 GB HBM each). Intra-node NPUs are fully interconnected through HCCS: each NPU connects to the other seven via seven 224-Gb/s HCCS links, yielding 196 GB/s of aggregate unidirectional intra-node bandwidth per NPU. Inter-node communication uses a 200 Gb/s NPU RoCE fabric. All end-to-end experiments use the GRPO algorithm, a maximum model length of 40960 tokens, and 16 samples per prompt. Following prior work[13, 39, 43, 63], experiments adopt the one-step asynchronous setting. While our evaluation focuses on GRPO, Libra is algorithm-agnostic and can be extended to other RL algorithms such as PPO[37].

Models. We conduct experiments on two widely adopted open-weight models: Qwen3-14B and Qwen3-30B-A3B (30B total parameters with 3B activated per token, a Mixture-of-Experts architecture) [53].

Baselines. We compare Libra against five representative baselines:

- *verl-Colocated* replicates the widely used execution mode adopted by existing RL frameworks [21, 40, 47, 55, 65]. In this mode, training and rollout share the same set of GPUs and execute alternately through the hybrid engine without explicit partitioning. All three *verl* baselines are deployed on the same verl codebase (v0.8.0) with identical Megatron-Core (v0.14.0) and vLLM (v0.9.2) backends as Libra.
- *verl-Static-Uniform* evenly splits the cluster into two halves: one for training and one for rollout. This configuration is adopted in a subset of experiments by recent disaggregated RL systems [39, 43].
- *verl-Greedy-Heuristic* minimizes training GPUs subject to memory and batch constraints, assigning remaining GPUs to high-TP rollout instances.
- *AReaL-Static-Optimal* represents AReaL [13, 31] using its official open-source codebase (v2.0.0), configured with the best static partition and parallelism strategy identified by Libra's cost evaluator (§4.5).
- *DynaRL* [49] dynamically shifts GPU counts between stages at runtime. We evaluate DynaRL using its official implementation in RLinf [56] (v0.3).

Workloads. We evaluate Libra on three representative agentic RL benchmarks: (1) *Search-R1* [25], an information-retrieval benchmark with multi-turn search interactions; (2) *R2E-Gym* [22], a software-engineering environment with 8.1K problems where agents invoke Bash/Python tools to edit code and verify patches; and (3) *DAPO-Math-17K* [57], a mathematical reasoning dataset of 17K competition-level problems.

7.1 End-to-End Experiments

Figure 7 compares Libra against five baselines across all workloads on the GPU (48× A800) and NPU (160× Ascend 910B3) clusters. Each column presents one workload, plotting throughput (left) and task reward versus wall-clock time (right).

Throughput. Libra consistently delivers the highest throughput across all benchmarks on both platforms. On the GPU cluster, Libra achieves 870–2,940 token/s, outperforming the strongest dynamic baseline DynaRL by 31%–44% (e.g., 2,900 vs. 2,000 token/s on Search-R1), AReaL-Static-Optimal by 57%–73%, and verl-Colocated by 246%–321% (3.5×–4.2× throughput speedup). On the NPU cluster, Libra sustains 1,100–3,900 token/s, exceeding DynaRL by 18%–37% and static/colocated baselines by 33%–233%.

Reward convergence. All methods reach comparable target rewards; the primary distinction is the wall-clock time required to get there (marked by the red dot for Libra in Figure 7). Libra finishes first on every benchmark on both clusters. Compared to DynaRL, Libra achieves 1.31×–1.45× speedups across the three benchmarks on GPUs (e.g., reaching target reward in 16.2 h vs. 23.5 h on Search-R1, and 54.0 h vs. 73.8 h on R2E-Gym) and 1.18×–1.37× speedups on NPUs (12.7 h vs. 17.4 h on Search-R1, and 42.7 h vs. 50.5 h on R2E-Gym). Against verl-Static-Uniform, Libra accelerates end-to-end training by up to 2.73× on GPUs and 2.26× on NPUs.

7.2 Effectiveness of Causality-Aware Routing

To isolate the impact of routing quality, we compare Libra's bucket scheduler against four alternatives on the Search-R1 workload with the same heterogeneous bucket configuration. *Oracle* routes each request using the ground-truth remaining sequence length known at runtime; it represents an unattainable upper bound since the true length is unavailable in advance. *Prediction-Based* employs the embedding-based length predictor from [38], which estimates sequence lengths from hidden-state embeddings. Under RL training, however, the LLM is continuously updated, causing the predictor's embeddings to drift and accuracy to degrade without periodic retraining. *MLFQ* adopts a reactive policy that migrates a request only after its length exceeds the current bucket's upper bound, inevitably incurring late and frequent migrations. *Load Balancing* distributes requests uniformly across buckets, ignoring sequence length.

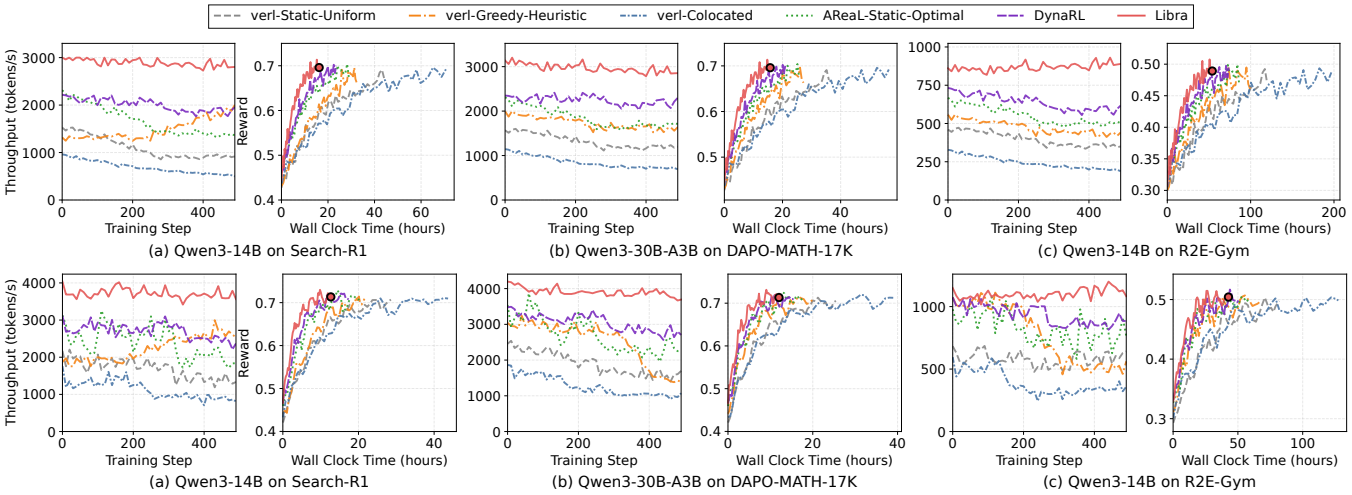


Figure 7. End-to-end training performance on three benchmarks. Each column shows throughput (left) and reward convergence (right) for one workload. Top row: GPU cluster; bottom row: NPU cluster.

Method	Per Decision Acc%	Migrate Ratio%*	Throughput (token/s)
Load Balancing	31.2	0	1502
MLFQ	44.8	108	1917
Prediction-Based	65.2	48.9	2289
Bucket Scheduler	91.1	8.2	2700
Oracle	100	0	3102

*Migration Ratio $= \frac{\text{total migrated tokens}}{\text{total tokens}}$.

Table 2. Comparison of routing policies.

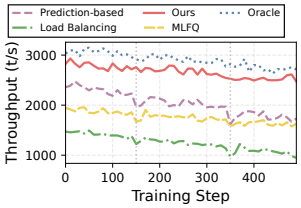


Figure 8. Ablation study of the scheduler.

Table 2 reports per-decision routing accuracy, migration ratio, and system throughput, while Figure 8 traces throughput evolution across training steps. Per-decision accuracy measures the probability that a scheduling algorithm makes a choice consistent with the ground-truth optimal bucket at each scheduling decision point. Causality-Guided Bucket Scheduler achieves 91.1% per-decision accuracy, closing most of the gap to the Oracle (100%) and substantially outperforming Prediction-Based (65.2%), MLFQ (44.8%), and Load Balancing (31.2%). As for system throughput: Libra delivers 2,700 tokens/s, outperforming Prediction-Based, MLFQ, and Load Balancing by 18%, 41%, and 80%, respectively. Figure 8 corroborates this trend: Libra sustains 2,500–2,800 tokens/s across training steps, closely tracking the Oracle curve and consistently exceeding the other policies by 400–1,000 tokens/s.

7.3 Ablation Study

Figure 9 quantifies the incremental throughput gains from progressively enabling Libra's four core components on R2E-Gym (Qwen3-14B). Panel (a) shows throughput evolution across training steps, and panel (b) presents the waterfall breakdown of average throughput contributions. Starting from the verl-Static-Uniform baseline (396 token/s), enabling the Planner with homogeneous TP raises throughput to 511 token/s (∼ 29.0%) by better balancing rollout and training. Heterogeneous TP support adds a further 57 token/s (∼ 11.2%). The Causality-Guided Bucket Scheduler contributes the largest gain, 162 token/s (∼ 28.5%), by routing requests to size-appropriate TP buckets. Finally, Elasticity yields a 140 token/s gain (∼ 19.2%), dynamically shifting idle workers to the bottleneck stage. In total, Libra reaches 870 token/s (∼ 120% above the baseline), matching the steady-state throughput reported in the end-to-end evaluation. Panel (a) also shows that Libra recovers fastest from workload drift, confirming that elasticity is essential for sustained performance under shifting conditions.

7.4 Overhead Analysis

Table 3 breaks down worker-state transition costs for Qwen3-14B on R2E-Gym. Switching from training to rollout incurs 15 ms teardown and 10.8 s vLLM activation; because weights reside on GPU and CUDA graphs are reused, this is an amortized one-time warm-up. The reverse transition (rollout→training) overlaps a 3.6 s snapshot with training, reloading state via RDMA in 4.4 s with $\lesssim$40 ms gradient and state alignment overhead. Crucially, transitions are triggered only during reconfiguration rather than every step. Across a 500-step Search-R1 run (Appendix D), hybrid transitions occurred only ∼26 times (once every 15–20 steps). Compared to the 116.6 s average step time, the 4.4 s reload is hidden behind active training passes, keeping cumulative transition overhead below 0.3% of the 16.2-hour wall-clock time. Furthermore, a heavier core partition rebuild was triggered

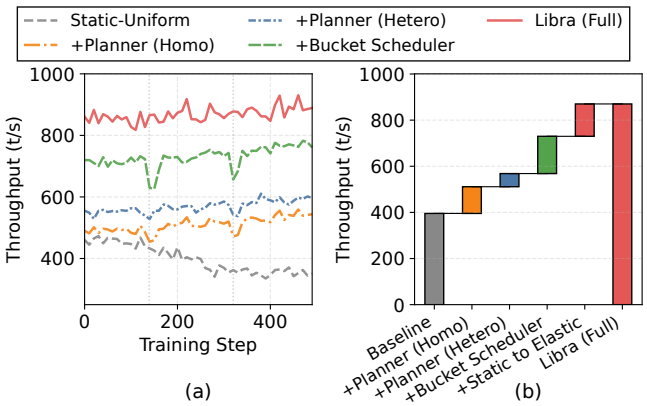


Figure 9. Ablation study on R2E-Gym (Qwen3-14B). (a) Throughput over training steps for progressively enabled Libra components. (b) Waterfall breakdown of average throughput gains from each component.

Table 3. Worker state transition breakdown in the Elastic Hybrid Pool.

Direction	Operation	Overhead	Mechanism
Training → Rollout	Training context teardown	15 ms	Release local resources
	vLLM instance activation	10.8 s	Enable CUDA graph replay; Reuse torch.compile cache
Rollout → Training	Snapshot capture	3.6 s	Per-step, overlapped with training
	State reload	4.4 s	Proceeds via RDMA
	Gradient channel setup	$\lesssim$20 ms	One-time registration
	Training state alignment	20 ms	Join via zero-gradient sync

only once at step 252, Libra deferred this rebuild to a natural pipeline slack period (while training was awaiting fresh rollout batches), the reconfiguration time was absorbed within the pipeline bubble.

Figure 10 quantifies remaining runtime overheads. Panel (a) shows KV-cache migration latency versus sequence length. Same-node transfer stays below 330 ms even at 40 K tokens, while cross-node transfer reaches 733 ms. Recomputation becomes cheaper than cross-node migration at longer sequences. Panel (b) measures planner search time as the cluster scales. Without memoization, search time rises to 12.3 s on 128 GPUs; with memoization it is capped at 1.9 s (~6.5× reduction), ensuring that re-planning remains a tiny fraction of the overall iteration time.

7.5 Cost Evaluator Fidelity

Figure 11 assesses the prediction fidelity of Libra's Cost Evaluator (CE). In panel (a), rollout step-time MAPE remains between 3.1% and 5.9% across sequence lengths and TP degrees. In panel (b), training iteration-time MAPE stays at 2.4–2.6% on Qwen3-14B and 5.0–5.5% on Qwen3-30B-A3B under heterogeneous micro-batch loads. In panel (c), CE predictions closely track 100 real iteration times with 6.35% average MAPE and 9.30% maximum error.

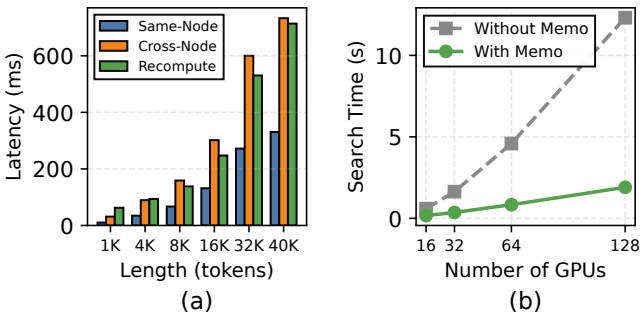


Figure 10. Overhead analysis. (a) KV cache migration latency across sequence lengths. (b) Planner search time vs. GPU count.

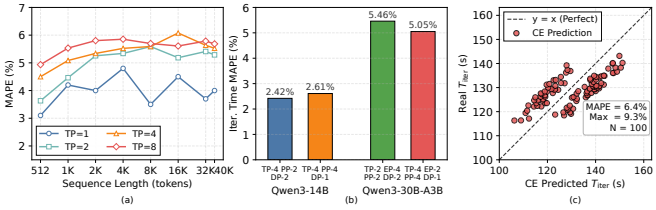


Figure 11. Cost Evaluator fidelity validation. (a) Rollout step-time prediction accuracy (MAPE). (b) Training iteration-time MAPE. (c) End-to-end scatter plot of CE-predicted versus measured iteration time over 100 estimations.

8 Related Work

Table 4 summarizes the comparison between Libra and representative related work.

RL frameworks for LLM post-training. Colocated frameworks [21, 40, 55, 65] alternate training and rollout on shared GPUs, while disaggregated frameworks [13, 15, 16, 19, 39, 46, 63] isolate stages onto dedicated clusters to execute asynchronously. Concurrent work DynaRL [49] dynamically moves GPUs across stages, but confines elasticity to 1D GPU counts without adapting internal parallelism strategies. Libra instead periodically re-plans both the cross-stage GPU allocation and each cluster's internal parallelism strategy, and elastically rebalances workers through a non-blocking protocol.

Rollout scheduling and long-tail mitigation. To mitigate long-tails, Heddle [59] and streamrl [63] employ auxiliary models for length prediction, which suffer representation drift under policy updates. Seer [35] and RhymeRL [20] rely on intra-group or cross-epoch response similarity, which fails when mid-trajectory tool returns heavily alter sequence length. Libra instead exploits runtime tool outcomes for causality-guided bucket routing.

Distributed parallelization and planning. Distributed pre-training managers [29, 32, 42, 61] optimize parallelism strategies for static workloads. However, they target single-phase training and do not consider the dynamic, coupled execution of RL rollout and training stages.

Table 4. Comparison with representative RL post-training and inference scheduling frameworks.

	Libra	DynaRL [49]	AReal [13]	Laminar [39]	Heddle [59]	veRL [40]
Cross-Stage Coordination						
Disaggregated Execution	✓	✓	✓	✓	–	–
Cross-Stage Resource Planning	✓	✓	×	×	×	×
Multi-Dim Parallelism Planning	✓	×	×	×	×	×
Non-Blocking Worker Migration	✓	×	×	×	×	×
Rollout Parallelism & Scheduling						
Length-Aware Scheduling	✓	×	×	×	–	×
Heterogeneous Parallelism	✓	×	×	×	✓	×

9 Conclusion

This paper presents Libra, an adaptive runtime for agentic RL post-training. Libra introduces a Causality-Guided Bucket Scheduler that routes requests across heterogeneous rollout buckets based on causal signals from tool-returns. Our evaluation on both GPU and NPU clusters demonstrates that Libra delivers substantial throughput gains and accelerated reward convergence. Libra has been open sourced.

References

[1] 2025. Megatron-LM. (2025). https://github.com/NVIDIA/Megatron-LM.

[2] Amey Agrawal, Nitin Kedia, Jayashree Mohan, Ashish Panwar, Nipun Kwatra, Bhargav S Gulavani, Ramachandran Ramjee, and Alexey Tumanov. 2024. Vidur: A Large-Scale Simulation Framework for LLM Inference. *Proceedings of Machine Learning and Systems* 6 (2024), 351–366.

[3] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *18th USENIX symposium on operating systems design and implementation (OSDI 24)*. 117–134.

[4] AMD ROCm. 2025. The vLLM MoE Playbook: A Practical Guide to TP, DP, PP and Expert Parallelism. https://rocm.blogs.amd.com/software-tools-optimization/vllm-moe-guide/README.html

[5] Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, Nicholas Joseph, Saurav Kadavath, Jackson Kernion, Tom Conerly, Sheer El-Showk, Nelson Elhage, Zac Hatfield-Dodds, Danny Hernandez, Tristan Hume, Scott Johnston, Shauna Kravec, Liane Lovitt, Neel Nanda, Catherine Olsson, Dario Amodei, Tom Brown, Jack Clark, Sam McCandlish, Chris Olah, Ben Mann, and Jared Kaplan. 2022. Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback. arXiv:2204.05862 [cs.CL] https://arxiv.org/abs/2204.05862

[6] Zhuohang Bian, Feiyang Wu, Teng Ma, and Youwei Zhuo. 2025. Tokencake: A KV-Cache-centric Serving Framework for LLM-based Multi-Agent Applications. *arXiv preprint arXiv:2510.18586* (2025).

[7] Haoyu Chen, Xue Li, Kun Qian, Yu Guan, Jin Zhao, and Xin Wang. 2026. Amoeba: Runtime Tensor Parallel Transformation for LLM Inference Services. arXiv:2509.19729 [cs.DC] https://arxiv.org/abs/2509.19729

[8] Paul F Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. 2017. Deep Reinforcement Learning from Human Preferences. *Advances in neural information processing systems* 30 (2017).

[9] Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Noveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, et al. 2025. Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities. *arXiv preprint arXiv:2507.06261* (2025).

[10] Jiazhan Feng, Shijue Huang, Xingwei Qu, Ge Zhang, Yujia Qin, Baoquan Zhong, Chengquan Jiang, Jinxin Chi, and Wanjun Zhong. 2025. ReTool: Reinforcement Learning for Strategic Tool Use in LLMs. arXiv:2504.11536 [cs.CL] https://arxiv.org/abs/2504.11536

[11] Yicheng Feng, Yuetao Chen, Kaiwen Chen, Jingzong Li, Tianyuan Wu, Peng Cheng, Chuan Wu, Wei Wang, Tsung-Yi Ho, and Hong Xu. 2024. Echo: Simulating Distributed Training At Scale. arXiv:2412.12487 [cs.LG] https://arxiv.org/abs/2412.12487

[12] Yicheng Feng, Xin Tan, Yangtao Deng, Yimin Jiang, Yibo Zhu, and Hong Xu. 2026. Frontier: Towards Comprehensive and Accurate LLM Inference Simulation. *arXiv preprint arXiv:2605.21312* (2026).

[13] Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, et al. 2026. Areal: A Large-Scale Asynchronous Reinforcement Learning System for Language Reasoning. *Advances in Neural Information Processing Systems* 38 (2026), 36256–36282.

[14] Wei Gao, Yuheng Zhao, Dakai An, Tianyuan Wu, Lunxi Cao, Shaopan Xiong, Ju Huang, Weixun Wang, Siran Yang, Wenbo Su, et al. 2025. RollPacker: Mitigating Long-Tail Rollouts for Fast, Synchronous RL Post-Training. *arXiv preprint arXiv:2509.21009* (2025).

[15] Wei Gao, Yuheng Zhao, Dilxat Muhtar, Dakai An, Xuchun Shang, Tianyuan Wu, Lunxi Cao, Shaopan Xiong, Weixun Wang, Ju Huang, Teng Ma, Siran Yang, Jiamang Wang, Lin Qu, Bo Zheng, and Wei Wang. 2026. ROSE: Rollout On Serving GPUs via Cooperative Elasticity for Agentic RL. arXiv:2605.06534 [cs.DC] https://arxiv.org/abs/2605.06534

[16] Wei Gao, Yuheng Zhao, Tianyuan Wu, Shaopan Xiong, Weixun Wang, Dakai An, Lunxi Cao, Dilxat Muhtar, Zichen Liu, Haizhou Zhao, Ju Huang, Siran Yang, Yongbin Li, Wenbo Su, Jiamang Wang, Lin Qu, Bo Zheng, and Wei Wang. 2025. RollArt: Scaling Agentic RL Training via Disaggregated Infrastructure. arXiv:2512.22560 [cs.DC] https://arxiv.org/abs/2512.22560

[17] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yujiu Yang, Minlie Huang, Nan Duan, Weizhu Chen, et al. 2024. Tora: A Tool-Integrated Reasoning Agent for Mathematical Problem Solving. In *International Conference on Learning Representations*, Vol. 2024. 48362–48395.

[18] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Honghui Ding, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jingchang Chen, Jingyang Yuan, Jinhao Tu, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaichao You, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingxu Zhou, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan

Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. 2025. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature* 645, 8081 (2025), 633–638. doi:10.1038/s41586-025-09422-z

[19] Zhenyu Han, Ansheng You, Haibo Wang, Kui Luo, Guang Yang, Wenqi Shi, Menglong Chen, Sicheng Zhang, Zeshun Lan, Chunshi Deng, et al. 2025. AsyncFlow: An Asynchronous Streaming RL Framework for Efficient LLM Post-Training. *arXiv preprint arXiv:2507.01663* (2025).

[20] Jingkai He, Tianjian Li, Erhu Feng, Dong Du, Qian Liu, Tao Liu, Yubin Xia, and Haibo Chen. 2025. History Rhymes: Accelerating LLM Reinforcement Learning with RhymeRL. arXiv:2508.18588 [cs.LG] https://arxiv.org/abs/2508.18588

[21] Jian Hu, Xibin Wu, Wei Shen, Jason Klein Liu, Zilin Zhu, Weixun Wang, Songlin Jiang, Haoran Wang, Hao Chen, Bin Chen, Weikai Fang, Xianyu, Yu Cao, Haotian Xu, and Yiming Liu. 2025. OpenRLHF: An Easy-to-use, Scalable and High-performance RLHF Framework. arXiv:2405.11143 [cs.AI] https://arxiv.org/abs/2405.11143

[22] Naman Jain, Jaskirat Singh, Manish Shetty, Liang Zheng, Koushik Sen, and Ion Stoica. 2025. R2E-Gym: Procedural Environments and Hybrid Verifiers for Scaling Open-Weights SWE Agents. arXiv:2504.07164 [cs.SE] https://arxiv.org/abs/2504.07164

[23] Dongfu Jiang, Yi Lu, Zhuofeng Li, Zhiheng Lyu, Ping Nie, Haozhe Wang, Alex Su, Hui Chen, Kai Zou, Chao Du, et al. 2025. VerlTool: Towards Holistic Agentic Reinforcement Learning with Tool Use. *arXiv preprint arXiv:2509.01055* (2025).

[24] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. Swe-bench: Can Language Models Resolve Real-World Github Issues?. In *International Conference on Learning Representations*, Vol. 2024. 54107–54157.

[25] Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. 2025. Search-R1: Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning. arXiv:2503.09516 [cs.CL] https://arxiv.org/abs/2503.09516

[26] Komal Kumar, Tajamul Ashraf, Omkar Thawakar, Rao Muhammad Anwer, Hisham Cholakkal, Mubarak Shah, Ming-Hsuan Yang, Phillip H. S. Torr, Fahad Shahbaz Khan, and Salman Khan. 2025. LLM Post-Training: A Deep Dive into Reasoning Large Language Models. arXiv:2502.21321 [cs.CL] https://arxiv.org/abs/2502.21321

[27] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th symposium on operating systems principles*. 611–626.

[28] Hanchen Li, Runyuan He, Qiuyang Mang, Qizheng Zhang, Huanzhi Mao, Xiaokun Chen, Hangrui Zhou, Alvin Cheung, Joseph Gonzalez, and Ion Stoica. 2026. Continuum: Efficient and Robust Multi-Turn LLM Agent Scheduling with KV Cache Time-to-Live. arXiv:2511.02230 [cs.OS] https://arxiv.org/abs/2511.02230

[29] Jiamin Li, Hong Xu, Yibo Zhu, Zherui Liu, Chuanxiong Guo, and Cong Wang. 2023. Lyra: Elastic Scheduling for Deep Learning Clusters. In *Proceedings of the Eighteenth European Conference on Computer Systems*. 835–850.

[30] MAA. 2025. American Invitational Mathematics Examination - AIME. https://huggingface.co/datasets/di-zhang-fdu/AIME_1983_2024

[31] Zhiyu Mei, Wei Fu, Kaiwei Li, Guangju Wang, Huanchen Zhang, and Yi Wu. 2025. ReaL: Efficient RLHF Training of Large Language Models with Parameter Reallocation. *Proceedings of Machine Learning and Systems* 7 (2025).

[32] Xupeng Miao, Yujie Wang, Youhe Jiang, Chunan Shi, Xiaonan Nie, Hailin Zhang, and Bin Cui. 2022. Galvatron: Efficient Transformer Training over Multiple GPUs Using Automatic Parallelism. *Proc. VLDB Endow.* 16, 3 (Nov. 2022), 470–479. doi:10.14778/3570690.3570697

[33] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. 2022. Training language models to follow instructions with human feedback. arXiv:2203.02155 [cs.CL] https://arxiv.org/abs/2203.02155

[34] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, and Soumith Chintala. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. doi:10.48550/arXiv.1912.01703

[35] Ruoyu Qin, Weiran He, Weixiao Huang, Yangkun Zhang, Yikai Zhao, Bo Pang, Xinran Xu, Yingdi Shan, Yongwei Wu, and Mingxing Zhang. 2025. Seer: Online Context Learning for Fast Synchronous LLM Reinforcement Learning. *arXiv preprint arXiv:2511.14617* (2025).

[36] Omkar Salpekar, Rohan Varma, Kenny Yu, Vladimir Ivanov, Yang Wang, Ahmed Sharif, Min Si, Shawn Xu, Feng Tian, Shengbao Zheng, Tristan Rice, Ankush Garg, Shangfu Peng, Shreyas Siravara, Wenyin Fu, Rodrigo de Castro, Adithya Gangidi, Andrey Obraztsov, Sharan Narang, Sergey Edunov, Maxim Naumov, Chunqiang Tang, and Mathew Oldham. 2026. Training LLMs with Fault Tolerant HSDP on 100,000 GPUs. arXiv:2602.00277 [cs.DC] https://arxiv.org/abs/2602.00277

[37] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. arXiv:1707.06347 [cs.LG] https://arxiv.org/abs/1707.06347

[38] Rana Shahout, Eran Malach, Chunwei Liu, Weifan Jiang, Minlan Yu, and Michael Mitzenmacher. 2025. Don't Stop Me Now: Embedding Based Scheduling for LLMs. In *International Conference on Learning Representations*, Vol. 2025. 63345–63368.

[39] Guangming Sheng, Yuxuan Tong, Borui Wan, Wang Zhang, Chaobo Jia, Xibin Wu, Yuqi Wu, Xiang Li, Chi Zhang, Yanghua Peng, Haibin Lin, Xin Liu, and Chuan Wu. 2025. Laminar: A Scalable Asynchronous RL Post-Training Framework. arXiv:2510.12633 [cs.LG] https://arxiv.org/abs/2510.12633

[40] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. 2025. HybridFlow: A Flexible and Efficient RLHF Framework. In *Proceedings of the Twentieth European Conference on Computer Systems* (Rotterdam, Netherlands) *(EuroSys '25)*. Association for Computing Machinery, New York, NY, USA, 1279–1297. doi:10.1145/3689031.3696075

[41] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 8634–8652. https://proceedings.neurips.cc/paper_files/paper/2023/file/1b44b878bb782e6954cd888628510e90-Paper-Conference.pdf

[42] Foteini Strati, Zhendong Zhang, George Manos, Ixeia Sánchez Périz, Qinghao Hu, Tiancheng Chen, Berk Buzcu, Song Han, Pamela Delgado, and Ana Klimovic. 2025. Sailor: Automating Distributed Training over Dynamic, Heterogeneous, and Geo-distributed Clusters. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles* (Lotte Hotel World, Seoul, Republic of Korea) *(SOSP '25)*. Association for Computing Machinery, New York, NY, USA, 204–220. doi:10.1145/

3731569.3764839

[43] Xin Tan, Yicheng Feng, Yu Zhou, Yimin Jiang, Yibo Zhu, and Hong Xu. 2026. OrchestrRL: Dynamic Compute and Network Orchestration for Disaggregated RL. arXiv:2601.01209 [cs.DC] https://arxiv.org/abs/2601.01209

[44] Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, Chuning Tang, Congcong Wang, Dehao Zhang, Enming Yuan, Enzhe Lu, Fengxiang Tang, Flood Sung, Guangda Wei, Guokun Lai, Haiqing Guo, Han Zhu, Hao Ding, Hao Hu, Hao Yang, Hao Zhang, Haotian Yao, Haotian Zhao, Haoyu Lu, Haoze Li, Haozhen Yu, Hongcheng Gao, Huabin Zheng, Huan Yuan, Jia Chen, Jianhang Guo, Jianlin Su, Jianzhou Wang, Jie Zhao, Jin Zhang, Jingyuan Liu, Junjie Yan, Junyan Wu, Lidong Shi, Ling Ye, Longhui Yu, Mengnan Dong, Neo Zhang, Ningchen Ma, Qiwei Pan, Qucheng Gong, Shaowei Liu, Shengling Ma, Shupeng Wei, Sihan Cao, Siying Huang, Tao Jiang, Weihao Gao, Weimin Xiong, Weiran He, Weixiao Huang, Weixin Xu, Wenhao Wu, Wenyang He, Xianghui Wei, Xianqing Jia, Xingzhe Wu, Xinran Xu, Xinxing Zu, Xinyu Zhou, Xuehai Pan, Y. Charles, Yang Li, Yangyang Hu, Yangyang Liu, Yanru Chen, Yejie Wang, Yibo Liu, Yidao Qin, Yifeng Liu, Ying Yang, Yiping Bao, Yulun Du, Yuxin Wu, Yuzhi Wang, Zaida Zhou, Zhaoji Wang, Zhaowei Li, Zhen Zhu, Zheng Zhang, Zhexu Wang, Zhilin Yang, Zhiqi Huang, Zihao Huang, Ziyao Xu, Zonghan Yang, and Zongyu Lin. 2025. Kimi k1.5: Scaling Reinforcement Learning with LLMs. arXiv:2501.12599 [cs.AI] https://arxiv.org/abs/2501.12599

[45] Hanyin Wang, Zhenbang Wu, Gururaj Kolar, Hariprasad Korsapati, Brian Bartlett, Bryan Hull, and Jimeng Sun. 2026. Reinforcement Learning for Out-of-Distribution Reasoning in LLMs: An Empirical Study on Diagnosis-Related Group Coding. *Advances in Neural Information Processing Systems* 38 (2026), 43031–43065.

[46] Taiyi Wang, Zhihao Wu, Jianheng Liu, Jianye Hao, Jun Wang, and Kun Shao. 2025. Distrl: An Asynchronous Distributed Reinforcement Learning Framework for On-Device Control Agent. In *International Conference on Learning Representations*, Vol. 2025. 74757–74782.

[47] Weixun Wang, Shaopan Xiong, Gengru Chen, Wei Gao, Sheng Guo, Yancheng He, Ju Huang, Jiaheng Liu, Zhendong Li, Xiaoyang Li, Zichen Liu, Haizhou Zhao, Dakai An, Lunxi Cao, Qiyang Cao, Wanxi Deng, Feilei Du, Yiliang Gu, Jiahe Li, Xiang Li, Mingjie Liu, Yijia Luo, Zihe Liu, Yadao Wang, Pei Wang, Tianyuan Wu, Yanan Wu, Yuheng Zhao, Shuaibing Zhao, Jin Yang, Siran Yang, Yingshui Tan, Huimin Yi, Yuchi Xu, Yujin Yuan, Xingyao Zhang, Lin Qu, Wenbo Su, Wei Wang, Jiamang Wang, and Bo Zheng. 2025. Reinforcement Learning Optimization for Large-Scale Learning: An Efficient and User-Friendly Scaling Library. arXiv:2506.06122 [cs.LG] https://arxiv.org/abs/2506.06122

[48] Xizheng Wang, Qingxu Li, Yichi Xu, Gang Lu, Dan Li, Li Chen, Heyang Zhou, Linkang Zheng, Sen Zhang, Yikai Zhu, et al. 2025. SimAI: Unifying Architecture Design and Performance Tuning for Large-Scale Large Language Model Training with Scalability and Precision. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. 541–558.

[49] Yuanqing Wang, Hao Lin, Junhao Hu, Chunyang Zhu, Quanlu Zhang, Zhen Guo, Yuchen Zhang, Xu Fu, Si Xu, Bo Dai, Zixiao Huang, Chao Yu, Boxun Li, Guohao Dai, Zhi Yang, and Yu Wang. 2026. DynaRL: Flexible and Dynamic Scheduling of Large-Scale Reinforcement Learning Training. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*. USENIX Association, 847–860.

[50] Bo Wu, Sid Wang, Yunhao Tang, Jia Ding, Eryk Helenowski, Liang Tan, Tengyu Xu, Tushar Gowda, Zhengxing Chen, Chen Zhu, et al. 2025. LlamaRL: A Distributed Asynchronous Reinforcement Learning Framework for Efficient Large-scale LLM Training. *arXiv preprint arXiv:2505.24034* (2025).

[51] Tian Xia, Hanchen Li, Zhifei Li, Xiaokun Chen, Hao Kang, Yifan Qiao, Yi Xu, and Ion Stoica. 2026. Idleness is Relative: Exploiting Tool-Call Idle Windows for Offloading in Agentic Systems with MORI. arXiv:2606.00866 [cs.OS] https://arxiv.org/abs/2606.00866

[52] Violet Xiang, Chase Blagden, Rafael Rafailov, Nathan Lile, Sang Truong, Chelsea Finn, and Nick Haber. 2025. Just Enough Thinking: Efficient Reasoning with Adaptive Length Penalties Reinforcement Learning. *arXiv preprint arXiv:2506.05256* (2025).

[53] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388* (2025).

[54] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *11th International Conference on Learning Representations, ICLR 2023*.

[55] Zhewei Yao, Reza Yazdani Aminabadi, Olatunji Ruwase, Samyam Rajbhandari, Xiaoxia Wu, Ammar Ahmad Awan, Jeff Rasley, Minjia Zhang, Conglong Li, Connor Holmes, et al. 2023. DeepSpeed-Chat: Easy, Fast and Affordable RLHF Training of ChatGPT-like Models at All Scales. *arXiv preprint arXiv:2308.01320* (2023).

[56] Chao Yu, Yuanqing Wang, Zhen Guo, Hao Lin, Si Xu, Hongzhi Zang, Quanlu Zhang, Yongji Wu, Chunyang Zhu, Junhao Hu, et al. 2026. RLinf: Flexible and Efficient Large-Scale Reinforcement Learning via Macro-to-Micro Flow Transformation. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*. 829–846.

[57] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao, Yonghui Wu, and Mingxuan Wang. 2025. DAPO: An Open-Source LLM Reinforcement Learning System at Scale. arXiv:2503.14476 [cs.LG] https://arxiv.org/abs/2503.14476

[58] Guibin Zhang, Hejia Geng, Xiaohang Yu, Zhenfei Yin, Zaibin Zhang, Zelin Tan, Heng Zhou, Zhongzhi Li, Xiangyuan Xue, Yijiang Li, et al. 2025. The Landscape of Agentic Reinforcement Learning for LLMs: A Survey. *arXiv preprint arXiv:2509.02547* (2025).

[59] Zili Zhang, Yinmin Zhong, Chengxu Yang, Chao Jin, Bingyang Wu, Xinming Wei, Yuliang Liu, and Xin Jin. 2026. Heddle: A Distributed Orchestration System for Agentic RL Rollout. arXiv:2603.28101 [cs.LG] https://arxiv.org/abs/2603.28101

[60] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, et al. 2023. PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel. *Proceedings of the VLDB Endowment* 16, 12 (2023), 3848–3860.

[61] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P Xing, et al. 2022. Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 559–578.

[62] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 193–210.

[63] Yinmin Zhong, Zili Zhang, Xiaoniu Song, Hanpeng Hu, Chao Jin, Bingyang Wu, Nuo Chen, Yukun Chen, Yu Zhou, Changyi Wan, Hongyu Zhou, Yimin Jiang, Yibo Zhu, and Daxin Jiang. 2025. StreamRL: Scalable, Heterogeneous, and Elastic RL for LLMs with Disaggregated Stream Generation. arXiv:2504.15930 [cs.LG] https://arxiv.org/abs/2504.15930

[64] Yinmin Zhong, Zili Zhang, Bingyang Wu, Shengyu Liu, Yukun Chen, Changyi Wan, Hanpeng Hu, Lei Xia, Ranchen Ming, Yibo Zhu, et al.

2025. Optimizing RLHF Training for Large Language Models with Stage Fusion. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. 489–503.

[65] Zilin Zhu, Chengxing Xie, Xin Lv, and slime Contributors. 2025. slime: An LLM post-training framework for RL Scaling. https://github.com/THUDM/slime. GitHub repository. Corresponding author: Xin Lv.

A Rationale of Rollout Cluster Parallelism Design

In Libra, the rollout cluster is designed with heterogeneous Tensor Parallelism (TP) as the primary parallelism strategy, while Pipeline Parallelism (PP) and Expert Parallelism (EP) are not explicitly included in the current rollout configuration. This appendix justifies this design choice from three perspectives: workload characteristics, problem orthogonality, and extensibility.

A.1 Workload Characteristics of Agentic RL Rollout

Agentic RL rollout is dominated by autoregressive decoding of long, variable-length trajectories. The key performance bottlenecks are (1) memory bandwidth and KV-cache pressure, especially for long sequences [3, 62], and (2) straggler effects caused by a small fraction of long-tailed trajectories [14, 35, 39, 59, 63, 64]. TP partitions model weights and the KV cache across GPUs, directly alleviating memory bottlenecks and balancing computation [7]. In contrast, PP splits model layers across devices, introducing sequential dependencies that do not help with stragglers and often amplify them: a single long request holding a pipeline stage blocks all downstream stages until completion [4].

A.2 Problem Orthogonality: Why TP Suffices for Long-Tailed Mitigation

Libra's core contribution is mitigating straggler effects in heterogeneous rollout. As shown in Figure 3, TP size directly determines the throughput trade-off: small TP (e.g., 1, 2) incurs low communication overhead and is ideal for short sequences, whereas large TP (e.g., 4, 8) offers better memory scalability and sustained throughput for long sequences.

PP does not offer this trade-off. It distributes layers sequentially, so a single long request still incurs high latency through all stages, and pipeline bubbles worsen under length imbalance. EP, while useful for Mixture-of-Experts (MoE) models, addresses expert load balancing rather than sequence-length-induced stragglers. Thus, PP and EP are orthogonal to Libra's primary optimization target.

A.3 Extensibility: How Libra Can Support PP and EP

Libra's heterogeneous rollout cluster design is not inherently closed to PP or EP. Extending the rollout cluster to support PP and EP follows the following principles.

Pipeline Parallelism in rollout. If a model is too large to fit within a single node, PP can be introduced inside each heterogeneous bucket as an additional intra-bucket dimension. The bucket then consumes $tp \times pp$ GPUs rather than tp GPUs. Extending the rollout DP (Algorithm 1) to support this requires two localized changes: (1) expanding the candidate set $\mathcal{T}$ to enumerate feasible (TP, PP) combinations with their corresponding total GPU counts, and (2) updating the CE interface to accept a PP argument so that the latency model accounts for inter-stage pipeline bubbles. Please note that vidur's latency model natively supports PP. The DP's optimal-substructure property remains unchanged because the state transition still consumes a fixed number of GPUs per bucket. The routing of Causality-Guided Bucket Scheduler can also be adapted to this setting, as the heterogeneous buckets retain their relative affinity for long and short sequences.

Expert Parallelism in rollout. For MoE models, EP can be transparent to Libra's planner because the inference engine handles it automatically. In vLLM, the engine automatically enables EP when the number of experts exceeds the TP size and is divisible by it, and to fall back to TP-only execution otherwise to avoid unnecessary all-to-all communication. Because Libra's heterogeneous rollout already traverses a range of TP configurations (e.g., TP=1, 2, 4, 8), each bucket implicitly covers both the EP-enabled and EP-disabled regimes: a smaller TP may trigger EP, while a larger TP may not. The planner therefore does not need to enumerate EP as an explicit dimension; extending rollout to MoE models amounts to letting the inference engine select EP automatically within each bucket. The inter-bucket scheduling mechanism remains unchanged.

B Offline Profile and Prefix-Tree Lifecycle

This appendix details the offline profile phase that precedes formal training, and provides empirical measurements on prefix-tree coverage and fallback frequency that complement the main text.

B.1 Offline Profile Phase

Before the first training iteration, Libra executes an *offline profile phase* whose primary goal is to obtain the initial sequence length distribution of the target workload. Using the initial policy model, Libra generates trajectories for the entire training corpus; the resulting length distribution then drives two downstream tasks: (1) calibrating the cost evaluator (CE) and deriving the initial resource allocation plan, and (2) bootstrapping the prefix tree with historical trajectory data.

CE calibration. The profile trajectories are fed to the CE to fit the per-operator polynomial latency models described in Section 4.5. The CE records the execution time of each layer type (Linear, Attention, etc.) at varying sequence lengths, yielding the coefficients γ, η, θ for the polynomial

$T(L)$. These models then drive the planner's initial DP search (Algorithm 1), producing the first rollout partition P_{rollout} and training strategy S_{train}.

Prefix-tree bootstrap. Every profile trajectory is inserted into the prefix tree following the procedure described in Phase 3 of Section 5. Consequently, every prompt in the training corpus is present in the tree from the very first training step. The profile phase produces $N \times K$ trajectories in total. These trajectories are inserted into the trie, which naturally shares prefixes: any two trajectories that follow the same tool-return sequence up to depth d share the path from the root to depth d. Consequently, the trie compactly represents the full history without materializing $N \times K$ independent branches. Table 5 shows the resulting tree statistics for the three benchmarks.

Table 5. Offline profile statistics and initial prefix-tree coverage.

Benchmark	#Prompts	Profile Time	Avg. Tree Depth
Search-R1	17,000	~1.24 h	5.2
R2E-Gym	8,135	~3.8 h	22.0
DAPO-Math-17K	17,000	~2.8 h	2.5

All profile experiments use Qwen3-14B on NVIDIA A800 80GB GPUs, deployed on 48 GPUs with TP-8 and DP-6, processing a batch of 1,024 prompts.

Reusability and Break-Even Analysis. Following standard methodology in distributed ML systems [2, 32, 61], the end-to-end training and convergence times reported in Section 7 evaluate active online training execution and do not include this upfront profiling phase. This exclusion is justified by the high reusability of the offline profile: because the profile depends strictly on the initial model weights and prompt corpus, a single profile is generated once and reused across all subsequent hyperparameter tuning runs (e.g., learning rates, KL penalties, reward scaling), algorithmic variations (e.g., GRPO vs. PPO), and ablation experiments on that benchmark.

Furthermore, even under the worst-case scenario where a profile is consumed by only a *single* training run, Libra achieves break-even rapidly due to its substantial end-to-end throughput advantages:

- On *Search-R1*, Libra saves 7.3 h of online training time compared to the strongest dynamic baseline DynaRL (16.2 h vs. 23.5 h). Factoring in the 1.24 h profiling cost, Libra achieves a net savings of 6.06 h (a 1.35× speedup), reaching the break-even point in under 20% of the training run.
- On *R2E-Gym*, Libra cuts training time by 19.8 h over DynaRL (54.0 h vs. 73.8 h), which easily amortizes the 3.8 h profile overhead and yields a net saving of 16.0 h (1.28× end- to-end speedup).

Once online training commences, Libra adapts to policy drift through sub-second runtime trie updates (<1 s, §B.2), completely eliminating the need to re-execute offline profiling during the training lifecycle.

B.2 Tree Update and Policy-Drift Adaptation

As RL training progresses, the policy model evolves and the trajectory distribution shifts. Libra updates the tree by inserting each completed trajectory along its return-state sequence (Phase 3 of Section 5), creating child nodes for unseen states as needed and recording the remaining length at every visited node. Consequently, subsequent lookups immediately benefit from the latest observations. To further adapt to policy drift and amortize stale statistics, Libra additionally rebuilds the tree offline at regular intervals (every T training steps, default $T = 50$) using the most recent trajectories. Because the tree is a lightweight lookup structure (a trie with scalar statistics per node), both incremental insertion and full rebuild take less than one second and incur no online overhead. This dual update mechanism is the primary means through which Causality-Guided Bucket Scheduler adapts to policy drift without retraining a parametric prediction model.

B.3 Fallback Frequency and Tree Coverage

A runtime routing lookup walks the tree from the root along the tool-return sequence observed so far. If the next return state has never been seen under the current prefix, the scheduler falls back to the parent node's statistics (Section 5). We instrumented Libra on the Search-R1 workload to measure how often this fallback occurs.

Table 6. Fallback frequency per training phase (Search-R1).

Training Phase	Avg. Tree Lookups	Avg. Fallbacks	Fallback Ratio
Steps 1–50	3,952	236	5.98%
Steps 51–100	4,248	174	4.10%
Steps 101–200	9,424	347	3.68%
Steps 201–400	16,492	462	2.80%

Table 6 shows that the fallback ratio decreases as training proceeds. During the initial phase (steps 1–50), the tree contains only the offline profile data, yet the fallback ratio is below 6%. As the policy explores new tool-return combinations, the tree is periodically rebuilt with fresh trajectories, and the fallback ratio drops to under 4% by the later stages of training. This empirically confirms that the offline profile provides sufficient initial coverage, and that periodic updates effectively amortize the sparse-revisit concern.

C Decoupled Communication Domain

This appendix details the implementation of Libra's decoupled communication domain, explains why membership

changes in the inter-replica plane do not block the core training pool.

C.1 RDMA-based All-Reduce Protocol Implementation

Libra realizes the inter-replica gradient exchange through a RDMA-based All-Reduce protocol inspired by FT-HSDP [36]. It departs from NCCL's static communicator model by adopting a hybrid control-plane/data-plane architecture.

Control plane (CPU async thread). The CPU thread manages all membership-related complexity. It maintains the active replica set through a consensus service, detects failures via heartbeat timeouts, and dynamically rewires the ring topology by updating per-rank RDMA send/receive buffer descriptors. Because these operations are lightweight metadata updates, they complete in milliseconds without touching GPU state.

Data plane (GPU stream). The GPU stream executes only data copy and reduction kernels using a ring algorithm (ReduceScatter followed by AllGather). To prevent GPU hangs, the kernel busy-polls a host-pinned ready flag with a software timeout rather than blocking indefinitely. When the flag signals data arrival, the kernel performs in-GPU reduction and forwards the result to the next neighbor.

C.2 Why the Core Pool Remains Unblocked

The decoupled design guarantees that the intra-replica NCCL collective never stalls for three reasons.

Reason 1: metadata-only membership updates. When a replica exits the core training pool, the CPU removes it from the inter-replica member list and recomputes the ring order. Core training workers are completely unaware of this change; the CPU merely refreshes neighbor-address tables in host-pinned descriptors before the next gradient-exchange launch. Unlike NCCL, which requires destroying and recreating the entire communicator (a multi-second operation), Libra's update is a microsecond-scale metadata edit.

Reason 2: ring algorithm tolerates absent nodes. The ring AllReduce algorithm does not require all participants to be simultaneously online. Once the exiting replica is excluded from the neighbor map, the data plane naturally skips it: each core training worker sends to and receives from its live predecessor and successor only. Because the GPU kernel derives its peer list from the CPU-updated metadata at launch time, it never issues RDMA operations to a dead endpoint and therefore never blocks on a timeout.

Reason 3: non-blocking catch-up. When a replica is ready to join the training process, the CPU inserts it into the ring at an arbitrary position and updates only its two immediate neighbors' routing tables. Core training workers refresh their RDMA address mappings at the next gradient-exchange boundary; this refresh is a single memory write visible to the GPU kernel within microseconds.

D Non-blocking Joining

This appendix formalizes the aggregation rule of the non-blocking joining protocol and proves that it preserves mathematical equivalence across both active elastic training and joining windows. We further analyze optimizer-state consistency under asynchronous recovery and provide empirical validation.

D.1 Aggregation Rule and Optimization Invariance

Workload model and fixed global batching. In agentic RL post-training (e.g., GRPO/PPO), each training step consumes a fixed global batch of rollout trajectories $\mathcal{B}$, comprising $S = |\mathcal{B}|$ samples (e.g., 4096 samples). The target optimization gradient is the exact sample-averaged gradient across this fixed batch:

$$\bar{g}^* = \frac{1}{S} \sum_{s \in \mathcal{B}} \nabla \ell(s; \theta). \tag{1}$$

When the training cluster operates with N_c core data-parallel replicas and N_h active hybrid replicas (total $N = N_c + N_h$ replicas), Libra partitions $\mathcal{B}$ disjointly across all live replicas:

$$\mathcal{B} = \left(\bigcup_{i=1}^{N_c} \mathcal{B}_i^{\text{core}} \right) \cup \left(\bigcup_{h=1}^{N_h} \mathcal{B}_h^{\text{hyb}} \right), \tag{2}$$

where $|\mathcal{B}_i^{\text{core}}| = S_i^{\text{core}}$ and $|\mathcal{B}_h^{\text{hyb}}| = S_h^{\text{hyb}}$ such that $\sum_{i=1}^{N_c} S_i^{\text{core}} + \sum_{h=1}^{N_h} S_h^{\text{hyb}} = S$. Adding hybrid DP replicas increases DP parallelism to accelerate the step by reducing each replica's local micro-batch count, without expanding the global batch size S.

Topology mapping and two-tier gradient aggregation. Each active hybrid DP replica $h \in \{1, \ldots, N_h\}$ is paired to a core DP replica $\pi(h) \in \{1, \ldots, N_c\}$. The control plane distributes hybrid replicas evenly across core replicas such that $|\mathcal{H}_i| \le \lceil N_h / N_c \rceil$, where $\mathcal{H}_i = \{h \mid \pi(h) = i\}$. Because each DP replica consists of $R_{\text{train}} = TP \times PP$ workers, this replica pairing π is physically established by configuring peer-to-peer RDMA side channels between corresponding workers (i.e., workers with matching intra-replica TP/PP positions) in hybrid replica h and core replica $\pi(h)$. Each replica k computes the unnormalized gradient sum over its assigned samples:

$$G_k = \sum_{s \in \mathcal{B}_k} \nabla \ell(s; \theta). \tag{3}$$

Hybrid DP replica h routes its gradient sum G_h^{hyb} to core DP replica $\pi(h)$ through the RDMA side channel. Core DP replica i accumulates external gradients into its local sum:

$$\tilde{G}_i = G_i^{\text{core}} + \sum_{h \in \mathcal{H}_i} G_h^{\text{hyb}}. \tag{4}$$

The core pool then executes an intra-core All-Reduce (Sum):

$$G_{\text{total}} = \sum_{i=1}^{N_c} \tilde{G}_i = \sum_{i=1}^{N_c} G_i^{\text{core}} + \sum_{h=1}^{N_h} G_h^{\text{hyb}} = \sum_{s \in \mathcal{B}} \nabla \ell(s; \theta). \tag{5}$$

The effective gradient for parameter updates is normalized by the fixed global sample count:

$$\bar{g} = \frac{G_{\text{total}}}{S}. \tag{6}$$

Mathematical equivalence across elastic states. This sample-normalized sum aggregation guarantees exact mathematical equivalence ($\bar{g} \equiv \bar{g}^*$) under both operational phases:

- *Active Elastic Training ($G_h \neq 0$):* From Eq. (5) and (6), $\bar{g}$ is strictly identical to $\bar{g}^*$. Because the normalization denominator S remains fixed, the gradient magnitude, effective sample weights, and loss landscape are strictly invariant to N_h. Hence, elastic transitions require no learning-rate rescaling or warmup adjustments.
- *Non-blocking Joining Window ($G_h = 0$):* During state restoration, a joining DP replica has not completed loading its snapshot, so it is assigned zero samples ($S_h^{\text{hyb}} = 0$) and emits a zero placeholder ($G_h^{\text{hyb}} = 0$). The global batch $\mathcal{B}$ is partitioned among the remaining live replicas (i.e., the N_c core replicas). In Eq. (4), $\tilde{G}_i = G_i^{\text{core}} + 0$. The All-Reduce sums to the exact total gradient over all S samples, and division by S yields the identical unbiased gradient $\bar{g}^*$.

D.2 Two-Phase Version Alignment and Optimizer Consistency

To eliminate state staleness, race conditions, or optimizer moment divergence when a hybrid DP replica joins without stalling ongoing training, Libra employs a deterministic two-phase version alignment protocol.

Phase 1: Intra-step snapshot reload. As quantified in Table 3, reloading the complete state snapshot (θ_t, m_t, v_t) from CPU host memory via RDMA requires only 4.4 s (executed concurrently across all constituent workers of the replica), which corresponds to ~3.8% of an active training step (average step time ~116.6 s). The joining replica initiates this reload at the onset of step t. By second 4.4, the replica reconstructs the exact model parameters θ_t and optimizer moments (m_t, v_t) (sharded according to the model-parallel configuration across its constituent workers). Crucially, throughout the remaining ~112 s of step t, the core replicas are engaged in forward and backward computation on their local micro-batches; their physical parameters and optimizer states remain strictly frozen at version t. Thus, the joining replica's restored state is perfectly concurrent with the core pool throughout step t, completely avoiding stale-parameter issues.

Phase 2: Lock-step step transition. Because the joining replica was loading state during the first 4.4 s of step t, it does not participate in step t's forward pass ($S_h = 0$) and emits a zero placeholder $G_h = 0$ via the side channel. When the core pool completes step t's All-Reduce, it computes the nominal global gradient $\bar{g}_t$ (Eq. 6) and forwards $\bar{g}_t$ to the joining replica through the side channel. Both the core replicas and the joining replica then execute the identical optimizer update on identical inputs $(\theta_t, m_t, v_t, \bar{g}_t)$. Under Adam, both sides compute:

$$m_{t+1} \leftarrow \beta_1 m_t + (1-\beta_1)\bar{g}_t, \tag{7}$$

$$v_{t+1} \leftarrow \beta_2 v_t + (1-\beta_2)(\bar{g}_t)^2, \tag{8}$$

$$\theta_{t+1} \leftarrow \theta_t - \eta \cdot \frac{m_{t+1}}{\sqrt{v_{t+1}} + \epsilon}. \tag{9}$$

Under plain SGD, this update simplifies to $\theta_{t+1} \leftarrow \theta_t - \eta\bar{g}_t$. Because the deterministic update functions and inputs are identical, parameters θ_{t+1} and optimizer moments (m_{t+1}, v_{t+1}) advance in lock-step to step $t+1$ without requiring full-state re-broadcasting. At the boundary of step $t+1$, the joining replica's state is mathematically identical to the core pool. From step $t+1$ onward, the replica operates as an active DP replica, computing non-zero gradients on its assigned batch partition.

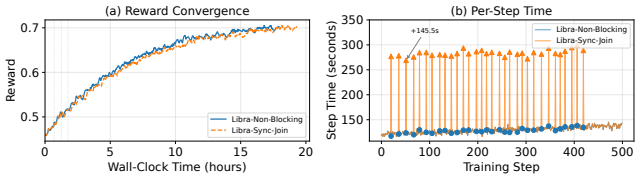


Figure 12. Validation of non-blocking joining on Search-R1. (a) Reward convergence over wall-clock time: Libra-Non-Blocking reaches the same reward faster than Libra-Sync-Join. (b) Per-step time: Libra-Sync-Join incurs a ~150 s spike at each transition step, while Libra-Non-Blocking remains flat.

D.3 Empirical Validation

To empirically validate that non-blocking joining does not degrade training convergence or parameter accuracy, we run a controlled paired experiment on the Search-R1 workload (Qwen3-14B, 500 training steps). Both configurations use identical planner decisions, heterogeneous TP buckets, and Causality-Guided Bucket Scheduler routing; the only difference is how worker transitions are handled.

Libra-Non-Blocking. The core training cluster continues advancing while joining replicas reload their snapshots asynchronously via the zero-gradient protocol described above.

Libra-Sync-Join. To emulate the behavior of conventional training frameworks (e.g., FSDP [34, 60], Megatron-LM [1]), whenever a hybrid DP replica transitions back to training, the entire core training cluster is torn down and rebuilt with the new replica included. This involves destroying the existing NCCL communicators, re-initializing the distributed training context, redistributing model and optimizer states, and re-warming the first training step.

Convergence and numerical equivalence. Figure 12(a) shows that Libra-Non-Blocking and Libra-Sync-Join reach the same final reward (0.70) after 500 training steps. To

strictly verify mathematical state equivalence beyond reward convergence, we tracked the maximum absolute parameter difference $\max |\theta_{\text{hyb}} - \theta_{\text{core}}|$ and optimizer moment differences across all model layers immediately following each joining transition. The difference remains below 1.2×10^{-7} (within FP32 numerical precision), confirming that non-blocking joining achieves exact state alignment without parameter drift or gradient distortion.

Overhead quantification. Figure 12(b) shows that Libra-Sync-Join incurs a ~150 s spike at every transition step, reflecting the full cost of global communicator rebuild and state redistribution.